SISTEMA COLABORATIVO DE ROBOTS MÓVILES PARA ASISTENCIA DE ORIENTACIÓN EN ENTORNOS INTERIORES CON MÚLTIPLES PISOS

Santiago Hernández Ávila

Jonny Alejandro Mejía León

Modalidad de Proyecto de Grado
(Proyecto de Investigación)

GED — Grupo de Estudio y Desarrollo en Robótica
(Robótica e Inteligencia Computacional)



UNIVERSIDAD SANTO TOMÁS
PRIMER CLAUSTRO UNIVERSITARIO DE COLOMBIA

UNIVERSIDAD SANTO TOMÁS
FACULTAD DE INGENIERÍA ELECTRÓNICA
PROGRAMA DE INGENIERÍA ELECTRÓNICA
Bogotá D.C.
2026

SISTEMA COLABORATIVO DE ROBOTS MÓVILES PARA ASISTENCIA DE ORIENTACIÓN EN ENTORNOS INTERIORES CON MÚLTIPLES PISOS

Santiago Hernández Ávila

Jonny Alejandro Mejía León

Proyecto de grado presentado como requisito para optar al título de:
Ingeniero Electrónico

DIRECTOR DEL PROYECTO
Ing. Armando Mateus Rojas, MSc.

UNIVERSIDAD SANTO TOMÁS
FACULTAD DE INGENIERÍA ELECTRÓNICA
PROGRAMA DE INGENIERÍA ELECTRÓNICA
Bogotá D.C.
2026

AUTORIDADES ACADÉMICAS

P. NOMBRES COMPLETOS, O.P.
Rector General

P. NOMBRES COMPLETOS, O.P.
Vicerrector Administrativo y Financiero General

P. NOMBRES COMPLETOS, O.P.
Vicerrector Académico General

NOMBRE COMPLETO
Secretaria General

P. NOMBRES COMPLETOS, O.P.
Director de División de Ingenierías

NOMBRE COMPLETO
Secretaria de División de Ingenierías

NOMBRE COMPLETO
Decano Facultad de Ingeniería Electrónica

Nota de aceptación:

_____

_____

_____

_____

_____

Firma del Director

_____

Firma del Jurado 1

_____

Firma del Jurado 2

Bogotá D.C., Colombia
octubre de 2026

ADVERTENCIA

La Universidad Santo Tomás no se hace responsable de las opiniones y conceptos expresados en el trabajo de grado, solo velará por qué no se publique nada contrario al dogma ni a la moral católica y porque el trabajo no tenga ataques personales y únicamente se vea el anhelo de buscar la verdad científica.

AGRADECIMIENTOS

Al ingeniero Armando Mateus Rojas, director del proyecto, por el seguimiento semanal y por insistir en que cada afirmación del documento tuviera detrás un dato que la sostuviera.

Al Grupo de Estudio y Desarrollo en Robótica de la Universidad Santo Tomás, por el espacio de trabajo y por los dos vehículos AWS DeepRacer. Sin ellos el proyecto habría tenido que empezar por construir el hardware.

A los docentes y compañeros de la Facultad de Ingeniería Electrónica que prestaron equipo, acompañaron las jornadas de medición en los pasillos y leyeron borradores de este documento.

A nuestras familias, por el respaldo durante los dos semestres que tomó el trabajo.

Conflicto de intereses

Los autores declaran no tener ningún conflicto de intereses. Los patrocinadores no participaron en el diseño del estudio, la recopilación, el análisis ni la interpretación de los datos publicados, ni en la redacción del documento.

RESUMEN

Orientarse en un edificio de varios pisos es difícil para quien no lo conoce, y el cambio de nivel corta la continuidad del recorrido. Las soluciones robóticas existentes resuelven ese salto de dos maneras: dotando al robot de mecanismos para subir escaleras, o conectándolo al ascensor. Las dos son caras y obligan a intervenir el edificio.

Este trabajo propone y evalúa una alternativa. Dos robots móviles terrestres, uno por planta, se reparten el guiado de una persona mediante un protocolo de relevo en la zona de escaleras. Ningún vehículo cruza de nivel. Un coordinador central asigna el agente según el nivel del origen y del destino. El usuario pide el servicio desde una interfaz web que funciona en cualquier teléfono, sin instalar nada y sin salida a internet.

La evaluación siguió un protocolo escrito antes de instrumentar el sistema, con cuatro métricas de definición operativa. En simulación, treinta misiones sorteadas dieron una campaña válida sin descartes, que permite afirmar con 95 % de confianza que la tasa de éxito supera el 70 %.

De diez misiones sorteadas sobre dos vehículos reales, uno por piso, nueve se completaron con relevo, con un tiempo total mediano de 87,6 s y la llegada a menos de 0,5 m del destino según la localización de cada vehículo. La décima se interrumpió porque el computador del coordinador se quedó sin batería. La precisión de llegada está limitada por la plataforma, que no se mueve por debajo de 0,40 m/s.

Palabras clave: robótica móvil colaborativa; sistemas multi-robot; navegación en interiores; protocolo de relevo; interacción humano-robot; ROS 2.

ABSTRACT

Finding one's way inside a multi-storey building is hard for first-time visitors, and changing floors breaks the continuity of the route. Existing robotic solutions address that gap in two ways: by equipping the robot to climb stairs, or by interfacing it with the building lift. Both are expensive and require modifying the infrastructure.

This work proposes and evaluates an alternative. Two ground mobile robots, one per floor, share the task of guiding a person through a handover protocol at the stairwell. Neither vehicle crosses between levels. A central coordinator assigns the agent according to the floor of the origin and the destination. The user requests the service from a web interface that runs on any phone, with no installation and no internet access.

The evaluation followed a protocol written before instrumenting the system, with four operationally defined metrics. In simulation, thirty randomised missions gave a valid campaign with no discards, which supports, with 95 % confidence, the claim that the success rate exceeds 70 %.

Of ten randomised missions on two real vehicles, one per floor, nine were completed with a handover, with a median total time of 87.6 s and arrival within 0.5 m of the goal according to each vehicle's own localisation. The tenth was interrupted when the coordinator's computer ran out of battery. Arrival accuracy is limited by the platform, which does not move below 0.40 m/s.

Keywords: collaborative mobile robotics; multi-robot systems; indoor navigation; handover protocol; human-robot interaction; ROS 2.

GLOSARIO

Los términos siguientes se usan desde el resumen y la introducción. El capítulo 2 los desarrolla y añade los demás.

Agente
: Cada uno de los dos robots del sistema. Se usa «agente» y no «robot» cuando importa su papel dentro de la coordinación y no su hardware.

AMCL
: Algoritmo de localización por filtro de partículas. Combina la odometría con las lecturas del láser para situar al robot dentro de un mapa que ya conoce. No construye el mapa: lo supone dado.

Banda muerta de tracción
: Rango de velocidades ordenadas en el que el vehículo no llega a moverse, porque la señal resultante no vence el rozamiento estático. En la plataforma de este trabajo abarca todo lo que está por debajo de 0,40 m/s, y condiciona la aproximación final del vehículo al destino.

Cinemática tipo automóvil (dirección Ackermann)
: Modelo de un vehículo que dirige con el eje delantero. Tiene radio de giro mínimo y no puede girar sobre su propio eje. Es lo contrario de la tracción diferencial, que es lo que la mayoría de las herramientas de navegación supone por omisión.

Continuidad del servicio
: Propiedad de no interrumpir la asistencia mientras el usuario cambia de nivel. Es el objeto de la pregunta problema.

Localización
: Estimación de la posición y la orientación del robot respecto a un marco de referencia fijo.

Mapa de ocupación
: Cuadrícula en la que cada celda se declara libre, ocupada o desconocida. Es la representación del edificio con la que funcionan la localización y la planificación.

Mapa de costos (*costmap*)
: Cuadrícula derivada de la anterior en la que cada celda lleva un costo de tránsito en lugar de una sola etiqueta. Permite que el planificador prefiera el centro del pasillo sin prohibir los bordes.

Middleware robótico
: Capa de software que comunica entre sí los módulos del sistema. Aquí es ROS 2.

Misión

Una solicitud de guiado completa, desde que el usuario elige origen y destino hasta que el sistema la cierra. Es la unidad de medida de la campaña experimental: las métricas se reportan por misión.

Observabilidad

Propiedad de una magnitud física que permite estimarla a partir de las mediciones disponibles. Si en cierta combinación de entorno y sensor una magnitud es inobservable, ningún ajuste de parámetros la recupera, porque la información no está en la señal. La sección 2.3.4 muestra un caso concreto en un pasillo recto.

Odometría

Estimación del desplazamiento propio del robot a partir de sus propias mediciones. La plataforma de este trabajo no tiene encoders, así que la obtiene comparando barridos sucesivos del láser.

Punto de transferencia

Lugar físico, en este trabajo la escalera, donde un agente entrega el guiado y el otro lo recibe.

Relevo

Traspaso de la responsabilidad de guiar a una persona, de un agente al otro, en un punto acordado de antemano. Es el aporte que el proyecto declara, y lo que sustituye al hardware de transición vertical.

Tasa de éxito

Proporción de misiones que terminan como el protocolo experimental declara que deben terminar. Su definición operativa está en la sección 3.5.

Tiempo de respuesta

Intervalo entre la solicitud del usuario y el instante en que el robot asignado arranca la tarea.

Zona de transición

Punto del edificio donde se cambia de nivel: escalera o ascensor. En este sistema ningún robot la atraviesa.

ABREVIATURAS

AMCL

Adaptive Monte Carlo Localization — localización adaptativa por método de Monte Carlo.

AWS

Amazon Web Services, fabricante de la plataforma DeepRacer empleada como vehículo.

CONPES

Consejo Nacional de Política Económica y Social (Colombia).

DDS

Data Distribution Service — protocolo de transporte sobre el que ROS 2 implementa sus comunicaciones.

GED

Grupo de Estudio y Desarrollo en Robótica, grupo de investigación en el que se inscribe el proyecto.

GUI

Graphical User Interface — interfaz gráfica de usuario.

HRI

Human-Robot Interaction — interacción humano-robot.

ICONTEC

Instituto Colombiano de Normas Técnicas y Certificación.

IMU

Inertial Measurement Unit — unidad de medición inercial.

JSON

JavaScript Object Notation, formato de intercambio de datos en texto.

MRS

Multi-Robot System — sistema multi-robot.

Nav2

Navigation2, marco de navegación autónoma de ROS 2.

NTC

Norma Técnica Colombiana.

PID

Control proporcional-integral-derivativo.

PWM

Pulse Width Modulation — modulación por ancho de pulso.

QoS

Quality of Service — políticas de entrega de un canal de comunicación.

ROS

Robot Operating System. La versión empleada es ROS 2, que no es una actualización de ROS 1 sino un rediseño sobre DDS.

SLAM

Simultaneous Localization and Mapping — localización y mapeo simultáneos.

CONTENIDO

Autoridades académicas I

Agradecimientos IV

Resumen V

Abstract VI

Glosario VII

Abreviaturas IX

Contenido XI

Lista de figuras XIV

Lista de tablas XVI

INTRODUCCIÓN 1

1. Contextualización 3

1.1. Planteamiento del problema . . . 3

1.2. Pregunta problema . . . 3

1.3. Objetivos . . . 4

1.3.1. Objetivo general . . . 4

1.3.2. Objetivos específicos . . . 4

2. Marco referencial 5

2.1. Antecedentes . . . 5

2.1.1. Robots que guían a personas . . . 5

2.1.2. Navegación autónoma en interiores . . . 6

2.1.3. Coordinación entre varios robots . . . 6

2.1.4. Movilidad entre pisos . . . 7

2.1.5. Aporte de este trabajo frente a los antecedentes . . . 7

2.2. Marco conceptual . . . 8

2.3. Marco teórico . . . 10

2.3.1. Asignación de tareas en sistemas multirrobot . . . 10

2.3.2. ROS 2 y la comunicación entre los componentes . . . 11
2.3.3. Cinemática tipo automóvil, planificación y seguimiento de trayectorias . . 12
2.3.4. Localización, odometría y observabilidad . . . 13
2.3.5. Simulación del sistema . . . 14
2.3.6. La interfaz web como cliente del sistema . . . 14
2.3.7. Evaluación experimental . . . 15
2.4. Marco legal . . . 16
2.4.1. Alcance de la operación y responsabilidad . . . 16
2.5. Marco ambiental . . . 17

3. Desarrollo **18**
3.1. Recorrido del trabajo . . . 19
3.1.1. La plataforma DonkeyCar y su simulador . . . 19
3.1.2. El modelo de AWS DeepRacer y su migración a ROS 2 Humble . . . 20
3.1.3. Los primeros entornos simulados y el seguidor de paredes . . . 21
3.1.4. La integración en simulación . . . 21
3.1.5. Del vehículo físico a la campaña con los dos vehículos . . . 22
3.2. Arquitectura funcional del sistema . . . 22
3.2.1. Requisitos . . . 22
3.2.2. El contrato de interfaces . . . 24
3.2.3. El planificador, escrito sin ROS . . . 24
3.2.4. La asignación de tareas . . . 25
3.2.5. Escenarios de operación en simulación . . . 26
3.2.6. De la simulación a los vehículos . . . 27
3.3. Plataforma robótica . . . 29
3.3.1. Componentes del vehículo . . . 29
3.3.2. Nav2 sobre un vehículo que no gira sobre su eje . . . 29
3.3.3. La odometría del láser como limitación principal . . . 30
3.3.4. Los mapas, derivados de medidas y no de SLAM . . . 30
3.3.5. La banda muerta de tracción . . . 32
3.3.6. La unidad de medición inercial de la tarjeta . . . 33
3.3.7. Los dos vehículos en la misma red . . . 33
3.3.8. La media vuelta en varios tiempos . . . 34
3.4. Interfaz de interacción con el usuario . . . 34
3.4.1. Una página web, no una aplicación . . . 34
3.4.2. Lo que el usuario ve . . . 35

3.4.3. Confirmación de cambio de piso . . . 35
3.4.4. Cancelación . . . 37
3.4.5. Ajustes tras las pruebas con los vehículos . . . 38
3.5. Protocolo experimental e instrumentación . . . 38
3.5.1. Definiciones operativas . . . 39
3.5.2. Diseño de la campaña en simulación . . . 40
3.5.3. Instrumentación . . . 40
3.5.4. Pruebas de verificación de la demostración física . . . 41

4. Resultados **42**
4.1. Evaluación en simulación . . . 42
4.1.1. Tasa de éxito . . . 42
4.1.2. Causa común de los cuatro fallos . . . 43
4.1.3. Tiempo de respuesta y tiempo de asignación . . . 43
4.2. Evaluación con los vehículos . . . 43
4.2.1. La odometría del vehículo . . . 44
4.2.2. La navegación y la precisión de llegada . . . 45
4.3. Evaluación del sistema completo con los dos vehículos . . . 46
4.3.1. Condiciones de las pruebas . . . 46
4.3.2. La misión con relevo . . . 46
4.3.3. La campaña con las misiones sorteadas . . . 47
4.3.4. El relevo entre los dos vehículos . . . 48
4.3.5. Criterio de conteo e incidencias . . . 49
4.4. Limitaciones de la medición . . . 49
4.4.1. Naturaleza del criterio de continuidad entre niveles . . . 49
4.4.2. Espacio desconocido marcado como libre en el mapa . . . 50
4.4.3. La llegada en los vehículos, sin medida externa . . . 50

CONCLUSIONES **51**

TRABAJOS FUTUROS **53**

Referencias bibliográficas **55**

LISTA DE FIGURAS

3.1. Exploración de la plataforma DonkeyCar. A la izquierda, su simulador conducido desde la interfaz web de la plataforma, con la imagen de la cámara del vehículo. A la derecha, el modelo propio del vehículo en el visualizador de ROS 2, con el chasis, las ruedas y la dirección. 19

3.2. El modelo del DeepRacer migrado a ROS 2 Humble. A la izquierda, en un mundo vacío, con el barrido del láser en azul. A la derecha, dentro del primer modelo del pasillo de la universidad, de 7 por 3 m; el sector sin barrido detrás del vehículo es el que no cubría el láser del modelo simulado. 20

3.3. Arquitectura del sistema en simulación, con un proceso de Gazebo por piso. Ningún robot atraviesa la zona de transición: lo que la cruza es el usuario, y lo que se traspasa entre agentes es la responsabilidad de guiarlo. 23

3.4. Misión con relevo en simulación. Cada ventana es un proceso de Gazebo con un piso y su vehículo; el láser de cada uno se ve en azul. La terminal muestra el resultado de la acción: éxito, un relevo y 100,1 s de duración. 26

3.5. Arquitectura del sistema en la campaña con los dos vehículos, en los pisos 3 y 4. La estructura es la de la figura 3.3; cambia la implementación de los bloques. 27

3.6. Tres recorridos en el hall del piso 2. El círculo es la pose que reporta la localización al terminar; el cuadrado, la posición real medida con flexómetro. La separación entre los dos es el error, y su causa está en la odometría, no en el algoritmo de localización. 31

3.7. Mapa de navegación del piso 4, derivado del levantamiento con flexómetro. El pasillo corre 25 m del extremo norte al sur; los salones quedan al este, y el ascensor y la escalera al oeste. 32

3.8. Interfaz durante una misión en simulación: el destino elegido, el botón de cancelar y el panel de estado del primer tramo, con el texto del coordinador, el vehículo activo y la distancia restante. 36

3.9. Interfaz en la etapa de espera de confirmación, durante su prueba automática: el estado de la misión lo publica la prueba, con el texto «Prueba de etapa 7», y la página muestra el botón «Ya estoy en el otro piso», de la versión anterior a los ajustes de la sección 3.4.5. 37

4.1. Una misión de la campaña en simulación. Se superponen el primer plan que calcula el navegador, la pose estimada a lo largo del recorrido y la odometría cruda. La separación creciente entre las dos últimas es la deriva que la localización corrige contra el mapa. 44

4.2. Cadena de tres tramos en el piso 4 y regreso a la salida: recorrido según la localización, llegadas medidas con flexómetro, la llegada estimada al Salón 403 y la pose de la localización al terminar cada tramo. 45
4.3. Duración de cada etapa en las nueve misiones de la campaña con los vehículos, según las marcas de etapa del coordinador. El tramo rayado es la espera de la confirmación de cambio de piso, contada desde que el segundo vehículo está en su escalera. 48

LISTA DE TABLAS

2.1. Líneas de antecedentes, lo que aportan a este trabajo y el supuesto que este trabajo cambia. 8

3.1. Etapas del trabajo frente a las fases del anteproyecto. 18
3.2. Etapas de una misión, tal como las publica el coordinador. 25
3.3. Cambios en la implementación al pasar de la simulación a los vehículos. 28
3.4. Componentes de cada vehículo y del computador central. 29
3.5. Definición operativa de las cuatro métricas del anteproyecto. 39
3.6. Pruebas de verificación de la demostración física y su estado al cerrar la toma de datos. 41

4.1. Tasa de éxito de la campaña en simulación, con intervalo de confianza de Wilson al 95 %. 42
4.2. Tiempo de respuesta de las treinta misiones, en escalones del reloj de la simulación. 43
4.3. Fracción del avance real que registra la odometría, por entorno. El criterio de la prueba es un error de 10 % o menos. 44
4.4. Cadena de tres tramos en el piso 4 y regreso a la salida: llegada de cada tramo. . . . 45
4.5. Campaña con los dos vehículos: las nueve misiones sorteadas que corrieron, todas completadas con un relevo. La llegada es la de la localización del vehículo, con una tolerancia de 0,5 m. 47

INTRODUCCIÓN

Orientarse dentro de un edificio grande no es igual de fácil para todo el mundo. Quien llega por primera vez construye un mapa mental a partir de información parcial, y lo va corrigiendo mientras camina. Ese esfuerzo crece cuando el espacio tiene pasillos repetidos, pocos puntos de referencia o señalización escasa. Y crece más cuando hay que cambiar de piso, porque la escalera corta la continuidad de lo que la persona lleva recorrido. Jiménez Muñoz lo señala al construir una arquitectura cognitiva que mete al usuario dentro de un sistema de mapas de varios niveles: la escalera es el punto donde la representación mental del recorrido deja de servir [1].

La robótica móvil colaborativa lleva años resolviendo tareas parecidas. El reparto de trabajo entre varios robots tiene base teórica desde que Gerkey y Matarić [2] formalizaron la asignación de tareas en sistemas multi-robot. Y Varghese et al. [3] muestran que varios agentes pueden mapear y localizarse de forma cooperativa. Casi todos esos sistemas operan, sin embargo, en una sola planta.

El cambio de nivel se ha abordado por dos caminos, y los dos pasan por el hardware. Kim et al. [4] combinan un mapa por piso con un grafo topológico y hacen que el robot use el ascensor. Song et al. [5] van más lejos: automatizan ese uso instalando dispositivos conectados en el edificio. La otra opción es dotar al robot de mecanismos para subir escaleras. Ambas encarecen la plataforma y exigen intervenir la infraestructura.

Este trabajo propone otra. En vez de un robot que cruza de piso, se usan **dos robots, cada uno dedicado a un nivel**. El usuario sube por su cuenta y un segundo agente lo recibe arriba. El cambio de piso deja de ser un problema de locomoción y pasa a ser uno de coordinación. Dos agentes se reparten una tarea y se la pasan en un punto acordado. A ese traspaso lo llamamos *relevo*, y es el aporte que el proyecto declara.

La pregunta que organiza el trabajo es si ese esquema funciona. No basta con que los robots se muevan. Hay que demostrar que la asistencia no se interrumpe cuando el usuario cambia de nivel, y medirlo con criterios fijados antes de correr los experimentos. Por eso el proyecto dedica tanto espacio al protocolo experimental como a la implementación.

El sistema se construyó sobre dos vehículos AWS DeepRacer y software libre. La coordinación corre en un nodo central que habla con cada agente a través de ROS 2. Macenski et al. [6] lo describen como un rediseño sobre DDS, no como una actualización de su antecesor. El usuario pide el servicio desde una página web que funciona en cualquier teléfono, sin instalar nada y sin salida a internet. La

evaluación se hizo primero en simulación, con una campaña de treinta misiones sorteadas, y después sobre los vehículos reales en los pasillos del edificio, con una campaña de diez misiones sorteadas entre los pisos 3 y 4.

El documento sigue ese orden. El capítulo 1 plantea el problema y los objetivos. El 2 reúne el marco conceptual, teórico, legal y ambiental. El 3 describe cómo se construyó el sistema: primero en el orden en que se hizo, desde la plataforma que proponía el anteproyecto, y después por objetivo específico. El 4 presenta los resultados y, con el mismo énfasis, sus límites. Los capítulos 5 y 6 cierran con las conclusiones y las líneas que quedan abiertas.

1. CONTEXTUALIZACIÓN

1.1 Planteamiento del problema

Orientarse en un edificio desconocido exige construir y actualizar una representación mental del espacio. El proceso se complica cuando el entorno tiene visibilidad limitada, estructuras repetidas y varios puntos de decisión. Omer y Monteriu [7] describen esos espacios compartidos con personas como el escenario que más exige a un robot. Vale también para quien camina sin conocerlos.

Las edificaciones de varios pisos añaden una dificultad propia. El cambio de nivel rompe la continuidad espacial y obliga a integrar información vertical en el recorrido. Es el mismo problema que Kim et al. [4] resuelven para un robot, con un grafo topológico entre plantas.

La robótica móvil colaborativa ha avanzado en exploración, logística y cobertura distribuida. La asignación de tareas entre agentes tiene una formalización aceptada desde el trabajo de Gerkey y Matarić [2]. Pero la mayoría de esos desarrollos se evaluó en entornos de una sola planta, donde la segmentación vertical no es el problema central. Los casos de limpieza doméstica y de almacén que documenta Nair [8] son representativos de ese escenario plano.

Resolver el cambio de piso con un solo robot resulta caro. Exige mecanismos de locomoción capaces de subir escaleras, o intervenir el sistema de control del ascensor. Song et al. [5] lo hacen instalando dispositivos conectados en la propia cabina. Las dos opciones elevan el costo del hardware y obligan a modificar el edificio.

De ahí el planteamiento de este trabajo. El problema no es cómo hacer que un robot suba, sino **cómo lograr que dos robots sencillos se repartan el recorrido y se pasen al usuario sin que la asistencia se interrumpa**. La comunicación entre agentes sustituye al hardware de transición vertical.

1.2 Pregunta problema

> ¿Es viable mantener la continuidad de la asistencia de orientación a una persona que cambia de nivel en un edificio, mediante un esquema de relevo entre dos robots móviles dedicados cada uno a una planta, sin que ninguno de ellos atraviese la zona de transición

vertical?

La pregunta tiene dos mitades y las dos se responden en el capítulo 4. La primera es de funcionamiento: si el relevo ocurre. La segunda es de medida: con qué incertidumbre se puede afirmar.

1.3 Objetivos

1.3.1 Objetivo general

Implementar un sistema colaborativo de dos robots móviles terrestres para la guía coordinada y eficiente de asistencia a visitantes en entornos interiores con múltiples pisos, mediante comunicación inter-robot, asignación dinámica de tareas y una interfaz móvil de interacción usuario-robot.

1.3.2 Objetivos específicos

1. Modelar la arquitectura funcional del sistema colaborativo de dos robots móviles, definiendo requerimientos, escenarios de operación con múltiples pisos, estrategia de asignación de tareas y esquema de comunicación inter-robot.
2. Desarrollar una plataforma robótica móvil basada en dos vehículos tipo DonkeyCar, integrando los módulos de locomoción, sensado, procesamiento y comunicación necesarios para la operación colaborativa en entornos interiores.
3. Programar una interfaz móvil para interacción humano-robot (HRI) basada en la selección de localizaciones de interés de origen y destino.
4. Evaluar el desempeño del sistema mediante pruebas experimentales en un entorno interior controlado, utilizando métricas como tiempo de respuesta, tiempo de asignación de robot, tasa de éxito en la entrega de asistencia y continuidad del servicio entre niveles.

Estos cuatro objetivos organizan el resto del documento. El capítulo 3 dedica una sección a cada uno, y el 4 reporta lo que se midió para cada uno.

2. MARCO REFERENCIAL

Este capítulo sitúa el trabajo frente a lo que ya existe y fija el vocabulario y la teoría con que se lee el resto del documento. Los antecedentes terminan con lo que este proyecto aporta frente a ellos. Siguen los términos técnicos, el fundamento de cada decisión de diseño y el marco legal y ambiental del prototipo.

2.1 Antecedentes

La revisión se ordena en cuatro líneas: robots que guían a personas, navegación autónoma en interiores, coordinación entre varios robots y movilidad entre pisos. Cada una resuelve bien una parte del problema. La última subsección añade la distribución del cómputo y reúne lo que ninguna resuelve, que es lo que este trabajo aborda.

2.1.1 Robots que guían a personas

Burgard et al. [9] describen RHINO, un robot guía que operó de forma autónoma durante seis días en un museo en 1997. Integraba localización, construcción de mapas, evasión de obstáculos, planificación, interacción con los visitantes y una interfaz web para seguirlo a distancia. De ese trabajo se toma una idea vigente: guiar personas es un problema de integración tanto como de navegación, porque ninguna pieza sirve si las demás fallan. Del mismo grupo salió la localización de Monte Carlo [10], base del algoritmo de localización de los vehículos de este proyecto.

Guerreiro et al. [11] llevan la idea a la asistencia de personas ciegas. Su robot, CaBot, tiene forma de maleta y transmite la dirección por un mango que vibra; lo evaluaron con diez participantes ciegos. El trabajo muestra dos exigencias que también aparecen aquí: el robot tiene que adaptarse a una persona que camina a su lado, y la interfaz tiene que comunicar el estado del guiado sin exigir atención constante.

Li y Gurocak [12] abordan la seguridad de compartir el espacio con personas, modificando el algoritmo Bug con sensores de proximidad para reducir las colisiones con los trabajadores cercanos. En este proyecto la respuesta está en la operación: una persona acompaña a cada vehículo durante las maniobras, y el usuario puede cancelar la misión en cualquier momento. Jiménez Muñoz [1]

integra al usuario en el propio sistema de mapas, con representaciones semánticas y reconocimiento de personas, en una plataforma asistencial de varios niveles. Es el antecedente más próximo en intención, porque trata a la persona como parte del problema de navegación.

En los tres casos un mismo robot acompaña a la persona durante todo el recorrido. Ese supuesto es el que este trabajo cambia.

2.1.2 Navegación autónoma en interiores

Sojan et al. [13] integran percepción, localización y planificación sobre ROS 2 para desplazarse en interiores dinámicos, con un mapa plano y localización por filtro de partículas. Es la arquitectura que este proyecto toma como punto de partida en cada vehículo.

El sistema que implementa ese flujo en ROS 2 es Nav2. Macenski et al. [14] lo presentan como sucesor de la pila de navegación de la primera generación de ROS: organiza la tarea con un árbol de comportamiento y arranca cada servidor como un nodo de ciclo de vida. Lo validaron en un campus universitario, navegando entre estudiantes. Macenski et al. [15] comparan los algoritmos de planificación, control y localización que hoy ofrece el ecosistema. Las dos referencias sirvieron para elegir los componentes de la sección 2.3.3. Varghese et al. [3] llevan el problema a varios agentes que construyen un mapa compartido y mantienen consistente su localización, con la misma técnica bidimensional sobre ROS.

Otra línea resuelve la navegación con aprendizaje en lugar de planificación explícita: redes neuronales que aprenden a conducir solo con imágenes [16], a evadir obstáculos fusionando sensores [17] o a reaccionar combinando aprendizaje profundo y lógica difusa [18]. Este proyecto no sigue esa vía, porque exige datos etiquetados del edificio, que no existen, y porque con planificación clásica cada decisión del vehículo se puede inspeccionar cuando hay que explicar un fallo.

2.1.3 Coordinación entre varios robots

Las estrategias de coordinación revisadas se diferencian por el nivel en el que cooperan. Nair [8] comparten la información sensorial entre los robots para mejorar el desempeño del conjunto. Roszkowska et al. [19] proponen una estructura jerárquica que supervisa el desplazamiento simultáneo de varios agentes para prevenir conflictos. Rudrusamy et al. [20] resuelven el problema desde el canal de comunicación, con control por difusión y seguimiento visual. Xu et al. [21] aportan una arquitectura modular sobre ROS para asignación cooperativa de tareas, y Zhan et al. [22] estudian la resiliencia del sistema cuando un nodo falla.

El marco teórico de la asignación lo fijan Gerkey y Matarić [2], cuya taxonomía clasifica cualquier esquema de reparto según cuántas tareas acepta un robot, cuántos robots requiere una tarea y con cuánta anticipación se decide. La sección 2.3.1 usa esa taxonomía, y su extensión por Korsah et al. [23], para situar el esquema de este trabajo.

El antecedente más cercano es Srivastava et al. [24], que coordinan recogida y entrega entre varios robots mediante planificación con relevos. Allí lo que pasa de un agente a otro es un paquete; aquí es una persona. Un paquete puede esperar indefinidamente en el punto de transferencia, y una persona no. Por eso el relevo de este trabajo incluye una confirmación explícita del usuario al llegar al otro piso, con un aviso a los 60 s y un plazo máximo de 120 s. De alcance comparable, por ser también un proyecto de grado de ingeniería, es Piguave Toala y Ronquillo Manosalvas [25], un sistema multirrobot de código abierto sobre ROS 2. Según la revisión del anteproyecto, todos estos trabajos operan en un solo nivel físico.

2.1.4 Movilidad entre pisos

El cambio de nivel se ha abordado sobre todo desde la logística y la exploración. Kim et al. [4] combinan un mapa por piso con un grafo topológico que los conecta, e integran la interacción con el ascensor. Song et al. [5] automatizan el uso del elevador con dispositivos conectados instalados en el edificio. Luo et al. [26] deciden entre ascensor y escalera con aprendizaje por refuerzo jerárquico, según el tránsito que encuentra el robot. Chen et al. [27] descomponen el entorno en pisos y escaleras para extender las técnicas bidimensionales a varios niveles, y Gong et al. [28] incorporan conciencia de piso a la navegación hacia objetos sin entrenamiento previo del escenario.

Estas propuestas atacan la movilidad entre pisos desde la infraestructura, el modelado topológico y la decisión adaptativa, y comparten un supuesto: el robot cruza de nivel, subiendo o accionando el ascensor. Todas se orientan a entrega o exploración antes que a la asistencia de personas.

2.1.5 Aporte de este trabajo frente a los antecedentes

A esas líneas se suma la de dónde ejecutar el procesamiento. Kehoe et al. [29] revisan las arquitecturas que lo delegan a servidores remotos, Tahir y Parasuraman [30] las que lo acercan al borde de la red, y Malavolta et al. [31] miden el costo de esa delegación sobre WiFi en robots terrestres. Sus resultados llevaron a que la navegación corra completa a bordo de cada vehículo y solo la coordinación, que intercambia mensajes pequeños, se centralice en un computador portátil, sin depender de salida a internet. La sección 2.3.2 explica cómo se refleja esa distribución en la red.

La literatura cubre bien cada componente por separado. La tabla 2.1 resume lo que cada línea aporta a este proyecto y el supuesto que este proyecto no comparte.

Tabla 2.1. Líneas de antecedentes, lo que aportan a este trabajo y el supuesto que este trabajo cambia.

Línea	Lo que se toma de ella	Supuesto que no se comparte
Robots guía	La integración de navegación e interacción; la localización de Monte Carlo	Un solo robot acompaña todo el recorrido
Navegación en interiores	El flujo mapa, localización y planificación sobre ROS 2 (Nav2)	Ninguno: se adopta, adaptado a la cinemática del vehículo
Coordinación multirrobot	La taxonomía de asignación; la planificación con relevos	Lo que se traspasa puede esperar sin límite
Movilidad entre pisos	El mapa por piso unido por un punto de transición	El robot cruza de nivel
Distribución del cómputo	La navegación a bordo y la coordinación en un nodo central	—

Lo que no aparece en ninguna línea es la combinación que aquí se propone: resolver la discontinuidad entre pisos sin que ningún robot la atraviese. El recorrido se reparte entre dos agentes dedicados, cada uno a un nivel, y la persona pasa de uno a otro en la zona de transición. La diferencia con los trabajos de movilidad vertical es económica antes que algorítmica. Esos trabajos obtienen la continuidad con hardware, con mecanismos para escaleras o con intervención del ascensor. Este proyecto la obtiene con comunicación entre agentes, que es lo que los trabajos de coordinación ya resuelven, aplicada a un problema donde no se había usado.

2.2 Marco conceptual

Esta sección define los términos con que se lee el resto del documento, en las seis categorías del anteproyecto. Los que tienen un fundamento propio se desarrollan en el marco teórico (sección 2.3).

En navegación y percepción, la **navegación autónoma** es el desplazamiento de un robot sin intervención humana, y reúne tres capacidades: percibir el entorno, estimar dónde está y decidir hacia dónde moverse. Cada vehículo percibe con un **LiDAR**, un sensor láser que mide distancias en un plano, en 360° a su alrededor, y con una **unidad de medición inercial** o IMU, que mide la velocidad de giro y la aceleración. Como no tiene codificadores en las ruedas, su **odometría**

sale de comparar barridos sucesivos del láser, y **AMCL** la corrige contra un mapa conocido. Que la odometría tenga con qué estimar el avance depende del sitio. Para medirlo, el proyecto definió la **información de avance** de un barrido: la fracción de los rayos del LiDAR que inciden sobre superficies orientadas hacia la dirección de marcha. Solo esos rayos cambian de longitud cuando el vehículo avanza. El entorno se representa con un **mapa de ocupación**, que dice qué celdas están libres, y con **mapas de costos**, que dan a cada celda un costo de tránsito. La **zona de transición** es el lugar del edificio donde se cambia de nivel; en este sistema, la escalera.

En control y locomoción, el vehículo sigue un **modelo cinemático tipo automóvil**, o de **dirección Ackermann**: dirige con el eje delantero y tiene un **radio de giro mínimo**. No puede girar en el sitio, como sí lo hace un robot de tracción diferencial. Para invertir el rumbo en un pasillo angosto hace una **media vuelta en varios tiempos**, alternando marcha atrás y avance con la dirección a tope. Cada punto de una trayectoria en el que el vehículo invierte la marcha es una **cúspide**. Las órdenes llegan al motor y al servomotor de dirección como señales de **modulación por ancho de pulso** (PWM). La tracción funciona en lazo abierto, sin un **control PID** que regule la velocidad, y tiene una **banda muerta**: por debajo de 0,40 m/s el motor no mueve el vehículo. Al final de cada tramo, el **verificador de meta** decide si el vehículo llegó, comparando su posición con una tolerancia.

En comunicación y sistemas distribuidos, el **middleware robótico** del proyecto es **ROS 2**. Organiza el software en **nodos**, programas que cumplen cada uno una función, que se comunican por **tópicos** de publicación y suscripción, **servicios** de petición y respuesta y **acciones**. Una acción representa una tarea larga: tiene una meta, retroalimentación periódica y un resultado, y se puede cancelar. El nodo que la ofrece es el **servidor de acción** y el que la invoca, el **cliente**; el coordinador de este proyecto es cliente ante cada agente y servidor ante la interfaz. Por debajo, ROS 2 usa **DDS**, un estándar en el que los participantes de la red se encuentran solos por **descubrimiento**. Quién habla con quién lo deciden el **dominio**, un número que separa el descubrimiento; la **partición**, un filtro dentro de un dominio, y el **espacio de nombres**, un prefijo como `/robot1` que indica a qué vehículo pertenece cada tópico. Las políticas de entrega de cada canal se llaman **calidad de servicio**. El **puente *rosbridge*** traduce entre ROS 2 y mensajes JSON sobre WebSocket, y un **contenedor** es un entorno de ejecución aislado, con sus propias bibliotecas, que permite tener dos versiones de ROS 2 en un mismo computador.

En arquitectura del sistema, la **asignación de tareas** es el reparto de actividades entre los robots según su disponibilidad y su ubicación. Cuando una tarea cruza de un nivel a otro, el reparto toma la forma de un **relevo**: un agente entrega la responsabilidad de guiar al usuario y el otro la recibe en el **punto de transferencia**. La **máquina de estados de misión** describe en qué punto va cada misión, y sus estados se publican para la interfaz y para el registro. El **planificador puro** calcula los tramos de una misión a partir del catálogo de puntos y del par origen-destino. No depende del middleware ni

de ningún estado externo, y por eso se puede verificar con cientos de casos sin levantar el simulador. El **registro de misión** es un documento estructurado que se compone a partir de la grabación de cada ejecución y se valida contra un esquema; es la evidencia primaria del experimento.

En interacción humano-robot, la **interacción humano-robot** o HRI estudia el diseño y la evaluación de sistemas robóticos pensados para trabajar con personas [1]. Aquí toma la forma de una **interfaz gráfica** web desde la que el usuario hace la **solicitud de servicio**. La **interfaz responsiva de selección** permite buscar un destino escribiendo su nombre sin perder el catálogo completo, y la **retroalimentación literal** es el principio por el que la interfaz muestra tal cual el texto que redacta el coordinador.

En métricas y criterios de evaluación, las cuatro métricas son las del anteproyecto: el **tiempo de respuesta**, el **tiempo de asignación**, la **tasa de éxito** y la **continuidad de servicio**. El tiempo de asignación es el intervalo entre aceptar la solicitud y designar el agente que la atiende. Para medirlas se usan además el **intervalo de confianza de Wilson** y la **falsabilidad** de un criterio, que se exponen en la sección 2.3.7. Un **invariante estructural** es una propiedad que se cumple siempre por el diseño del sistema y que, por eso, no es un resultado experimental. La **cota superior por misión** es lo que se informa cuando el instrumento no tiene resolución para dar el valor exacto. Con los vehículos reales, la demostración se organizó en **pruebas de verificación**, condiciones acordadas con el director antes de correrlas, como que la odometría registre el avance con un error del 10 % o menos. Decidirlas exige una **verdad de terreno**, la medida independiente con que se contrasta el sistema, que en este trabajo es la distancia tomada con flexómetro.

2.3 Marco teórico

Cada subsección expone el fundamento de una parte del sistema y la literatura que lo respalda. El orden va de la coordinación, que es el aporte del trabajo, a la navegación de cada vehículo, y termina en los criterios con que se evaluó.

2.3.1 Asignación de tareas en sistemas multirrobot

Gerkey y Matarić [2] clasifican los problemas de asignación de tareas entre robots según tres ejes. El primero distingue robots que ejecutan una sola tarea a la vez (ST, *single-task*) de robots que atienden varias (MT). El segundo distingue tareas que requieren un solo robot (SR, *single-robot*) de tareas que requieren varios (MR). El tercero distingue la asignación instantánea (IA), que se decide al llegar cada tarea, de la extendida en el tiempo (TA), que planifica asignaciones futuras.

La asignación de este sistema es ST–SR–IA. Cada vehículo ejecuta un tramo a la vez, cada tramo lo ejecuta un solo vehículo, y el agente se elige al llegar la solicitud. Los mismos autores demuestran que esa clase es una instancia del problema de asignación óptima, que el método húngaro resuelve en tiempo polinómico [32]. En este problema, sin embargo, la matriz de costos tiene un solo candidato admisible por tarea, porque cada vehículo está confinado a su nivel y hay uno solo por nivel. El método húngaro devuelve siempre la correspondencia que el sistema guarda como tabla de configuración, en la que cada nivel tiene asignado su vehículo. La ausencia de un optimizador es, por tanto, una consecuencia de la estructura del problema y no una carencia de la implementación.

Por la misma razón se descartaron los mecanismos de mercado. En el protocolo de red de contratos [33], un gestor anuncia la tarea, los agentes ofertan su costo y la tarea se adjudica a la mejor oferta. MURDOCH [34] aplica ese esquema a robots sobre publicación y suscripción, y CBBA [35] lo lleva a una subasta descentralizada en la que los agentes llegan a consenso sin coordinador central. Con un solo agente elegible por tramo, cualquier subasta tiene un único postor: añade mensajes y latencia sin cambiar la decisión. CBBA, además, elimina el coordinador central, que la arquitectura mantiene por diseño.

La misión completa sí se sale de esa clase. El relevo impone una restricción de precedencia: el tramo del segundo vehículo no puede empezar hasta que el primero termine el suyo y el usuario cambie de piso. Las agendas de los dos agentes quedan acopladas, y la taxonomía de 2004 está construida para tareas independientes. Korsah et al. [23] la extienden según el grado de interdependencia entre tareas, y en su clasificación esta misión pertenece a la clase XD, de dependencias entre agendas (*cross-schedule dependencies*). Ahí está el aporte del trabajo: la asignación es trivial y óptima por construcción, y la coordinación del relevo pertenece a una clase de problema que la taxonomía clásica del área no contempla.

El análisis también delimita el alcance. El sistema atiende una misión a la vez y no tiene noción de agente ocupado ni cola de solicitudes. Con dos o más vehículos elegibles por tarea, o con misiones simultáneas, la matriz de costos deja de tener un único candidato, y la elección del agente pasa a depender del método de asignación. Esa extensión se plantea en los trabajos futuros.

2.3.2 ROS 2 y la comunicación entre los componentes

Todos los componentes se comunican mediante ROS 2, que Macenski et al. [6] describen como un rediseño sobre DDS, el estándar de comunicación de datos en tiempo real del Object Management Group [36]. DDS aporta el descubrimiento automático de los participantes y las políticas de calidad de servicio de cada canal.

El descubrimiento tiene un costo que crece con el tamaño del sistema. Por omisión, cada proceso se anuncia en la red y empareja sus publicadores y suscriptores con los de todos los demás, y sobre una red inalámbrica ese tráfico compite con la navegación por el procesador de cada vehículo (sección 3.3.7). ROS 2 Jazzy permite restringir el descubrimiento a los procesos del mismo computador y declarar una lista fija de equipos con los que sí se comunica. Con esas dos opciones la red adopta la topología en estrella del diseño. Dentro de cada vehículo los procesos se descubren entre sí, y hacia afuera solo se anuncia al computador portátil lo que el coordinador necesita, de modo que los vehículos no se ven entre sí.

Dos mecanismos más separan a los vehículos. La partición de comunicaciones deja dentro de cada vehículo los tópicos del software del fabricante, que no llevan espacio de nombres. Un espacio de nombres distinto para cada vehículo distingue la navegación, el estado y la media vuelta de cada agente cuando llegan al portátil. El sistema usa además dos distribuciones de ROS 2, Humble en simulación y Jazzy en los vehículos; la sección 3.2.6 explica por qué y qué cambia entre una y otra.

2.3.3 Cinemática tipo automóvil, planificación y seguimiento de trayectorias

Un vehículo con dirección Ackermann está sujeto a una restricción no holonómica: no puede desplazarse de lado ni girar sobre su propio eje, y su curvatura está acotada por el radio de giro mínimo [37]. Por eso no toda trayectoria libre de obstáculos es ejecutable, y un planificador que solo considera la posición propone giros que el vehículo no puede hacer.

Reeds y Shepp [38] resolvieron el problema geométrico de fondo: el camino más corto entre dos poses para un automóvil que puede avanzar y retroceder, con radio de giro mínimo. Basta considerar caminos formados por arcos de radio mínimo y segmentos rectos, con a lo sumo dos inversiones de marcha, y el óptimo está siempre en un conjunto finito de candidatos con fórmulas explícitas. Como el vehículo de este trabajo puede retroceder, ese es su modelo de movimiento.

Sobre un mapa con obstáculos hace falta además una búsqueda. Dolgov et al. [39] propusieron el A* híbrido: una búsqueda A* sobre la posición y la orientación del vehículo, en la que cada expansión respeta la cinemática y el camino resultante se suaviza después. Lo validaron en los estacionamientos del DARPA Urban Challenge de 2007, un escenario de maniobras con inversión de marcha parecido al de un pasillo angosto. Nav2 lo implementa en el planificador Smac, que admite el modelo Reeds-Shepp [40], y es el que usan los vehículos de este proyecto. El planificador trabaja sobre mapas de costos organizados por capas [41]: la estática viene del mapa, la de obstáculos, de las lecturas del sensor, y la de inflado extiende el costo alrededor de cada obstáculo según el tamaño del vehículo. El mapa de costos global sirve al planificador y el local, que cubre los alrededores del

vehículo, al controlador.

El controlador convierte la trayectoria en órdenes de velocidad. La persecución pura [42] elige sobre la trayectoria un punto situado a una distancia de anticipación ℓ del vehículo y calcula el arco de circunferencia que lleva hasta él. Llamando y a la distancia lateral del punto al eje del vehículo, la curvatura ordenada es

$$\kappa = \frac{2y}{\ell^2}. \tag{2.1}$$

No requiere un modelo dinámico y siempre ordena arcos que la dirección puede seguir. Macenski et al. [43] le añadieron una regulación de velocidad en las curvas cerradas y cerca de los obstáculos, y la marcha atrás, de modo que sigue las cúspides que propone el planificador.

Esa configuración tiene un límite cerca de la meta. En este trabajo la distancia de anticipación va de 0,3 a 0,9 m. Cuando una cúspide queda muy cerca de la meta, esa distancia se acorta por debajo de un umbral en el que una guarda de seguridad del propio controlador anula la curvatura. El vehículo sigue invirtiendo la marcha sin dirigir y maniobra decenas de veces en el mismo punto. Por eso la media vuelta se resolvió con una maniobra propia del agente (sección 3.3.8).

2.3.4 Localización, odometría y observabilidad

La localización de Monte Carlo representa la creencia sobre la pose del vehículo con un conjunto de partículas, cada una con un peso [10], [44]. En cada paso, el modelo de movimiento desplaza las partículas según la odometría. Después, el modelo de medición las pondera según cuánto se parece el barrido del láser a lo que se vería desde esa pose en el mapa, y un remuestreo las concentra en las poses más probables. AMCL es la variante que ajusta el número de partículas a la incertidumbre; en este trabajo, entre 500 y 2000. Su precisión depende de la calidad de la odometría entre una actualización y la siguiente.

Como los vehículos no tienen codificadores en las ruedas, la odometría sale del láser. Jaimez et al. [45] estiman el movimiento plano de un láser a partir de barridos consecutivos con un enfoque de flujo de distancias (*range flow*). Alinean los barridos de forma densa, usando sus gradientes y sin buscar correspondencias entre puntos, en alrededor de un milisegundo por barrido. Es el método que implementa el paquete rf2o, que usan los vehículos.

Todo estimador que se apoya en la geometría del entorno hereda una limitación. Zhang et al. [46] la formalizan como degeneración: hay direcciones del espacio de estados en las que las mediciones no restringen la solución. Un pasillo recto y de sección uniforme es el caso típico. Al avanzar a lo largo de su eje, cada barrido es casi idéntico al anterior y el estimador devuelve un desplazamiento

cercano a cero, coherente con lo que mide el sensor. La posición longitudinal es inobservable.

De ahí sale la diferencia entre dos tipos de error que se parecen. Un error de calibración se corrige ajustando parámetros; una inobservabilidad estructural no, porque la información no está en la señal. La información de avance, definida en la sección 2.2, mide cuánta hay en cada sitio. El criterio que el proyecto adoptó para elegir dónde operar es que el entorno ofrezca estructura no uniforme dentro del alcance del sensor durante todo el recorrido. La construcción de mapas comparte el límite: la herramienta de SLAM de ROS 2 [47] usa la misma odometría del láser, y en un pasillo uniforme no tiene con qué cerrar el mapa. Por eso los mapas de este proyecto se derivaron de un levantamiento con flexómetro (sección 3.3.4).

El rumbo tiene una solución distinta, porque el giroscopio de la IMU lo mide de forma directa, sin depender del entorno. Un filtro de Kalman extendido combina las dos fuentes, ponderando cada medición según su incertidumbre [44]. El paquete robot_localization lo implementa para ROS de forma general, permitiendo elegir qué componente de cada sensor se fusiona [48]. En este trabajo el filtro toma de la odometría láser las velocidades de avance y de la IMU la velocidad de giro. El rumbo mejora; la posición no, porque sigue saliendo del láser.

2.3.5 Simulación del sistema

La campaña de evaluación corrió en Gazebo 11 con ROS 2 Humble. Gazebo es un simulador de código abierto pensado desde su origen para varios robots, con motor de física, modelos de sensores e integración con ROS [49]. La cinemática Ackermann del vehículo se simula con un complemento propio del paquete de simulación del DeepRacer, en lugar del genérico de Gazebo. Los dos vehículos comparten un dominio de comunicaciones y se separan por espacio de nombres, incluido el marco de referencia de su mapa. Con la configuración por omisión, dar a cada vehículo un dominio distinto corta también el descubrimiento entre el coordinador y los agentes. El tiempo que ven los nodos avanza en saltos de 100 ms, y eso fija la resolución con que se miden los tiempos de la campaña (sección 4.4).

2.3.6 La interfaz web como cliente del sistema

Crick et al. [50] propusieron *rosbridge* para que programas ajenos a ROS, como un navegador, usen tópicos, servicios y acciones mediante mensajes JSON sobre WebSocket. La versión para ROS 2 expone las acciones con operaciones propias, que la biblioteca de cliente web más difundida no implementa, porque fue escrita para las acciones de la primera generación de ROS. Por eso la

interfaz de este trabajo es un cliente propio, sin dependencias externas, ya que debe funcionar sin salida a internet.

La interfaz no contiene lógica del dominio: el texto que lee el usuario en cada estado lo escribe el coordinador, y la interfaz lo muestra literal. Si el navegador interpretara el estado para decidir qué texto mostrar, duplicaría una lógica que ya vive en el planificador, y las dos copias acabarían divergiendo.

2.3.7 Evaluación experimental

La evaluación se apoya en las cuatro métricas del anteproyecto, cada una con una definición operativa que indica qué marca del registro se resta de cuál. El protocolo se escribió antes de instrumentar el sistema, y las misiones se sortean con una semilla fijada antes de correrlas, para que ni el instrumento ni los recorridos se elijan después de ver los resultados.

La tasa de éxito es una proporción estimada sobre una muestra pequeña, y para ella falla el intervalo de confianza de la aproximación normal. Brown et al. [51] mostraron que su cobertura real oscila de forma irregular, incluso con muestras grandes, y empeora con proporciones cercanas a 0 o a 1, y recomiendan el intervalo de Wilson [52]. Sea n el número de misiones, $\hat{p}$ la proporción observada de éxitos y z el cuantil de la confianza deseada ($z = 1{,}96$ para el 95 %). El intervalo de Wilson es

$$\frac{\hat{p} + \dfrac{z^2}{2n} \pm z\sqrt{\dfrac{\hat{p}(1-\hat{p})}{n} + \dfrac{z^2}{4n^2}}}{1 + \dfrac{z^2}{n}}. \tag{2.2}$$

Su ancho decide qué se puede afirmar con una muestra dada (sección 4.1).

El segundo principio es de método. Para Popper [53], una afirmación aporta conocimiento empírico solo si alguna observación posible podría refutarla. Trasladado a un experimento, un criterio de aceptación solo es evidencia si alguna ejecución del sistema puede incumplirlo. La sección 4.4 muestra un criterio del proyecto que no cumplía esa condición y cómo se reformuló.

La demostración sobre los vehículos reales sigue otra lógica. La norma ASTM F3244-21 [54] establece un método de prueba para la navegación de robots en un área definida, con repeticiones del recorrido entre marcas fijas en el piso. El número de repeticiones fija cuánto se puede afirmar: si las n repeticiones salen bien, la fiabilidad mínima que se demuestra con una confianza C es

$$R = (1 - C)^{1/n}, \tag{2.3}$$

que con $n = 10$ y $C = 0{,}85$ da $R \approx 0{,}83$. Con las cinco a diez repeticiones que caben en el calendario, la campaña física demuestra que el sistema opera sobre los vehículos, pero no estima su tasa de éxito con la precisión de la simulación. Por eso las dos campañas se reportan por separado: la de simulación produce la evidencia estadística y la física, la demostración.

2.4 Marco legal

El proyecto se desarrolla dentro de la política pública colombiana de transformación digital y de ciencia, tecnología e innovación, que enmarcan dos documentos. El CONPES 3975 [55] fija la Política Nacional para la Transformación Digital e Inteligencia Artificial. El CONPES 4069 [56] establece la Política Nacional de Ciencia, Tecnología e Innovación. Ambos piden a la academia que desarrolle prototipos con herramientas de la Cuarta Revolución Industrial, entre ellas la robótica autónoma, y que lo haga con diseño local e impacto social.

Este trabajo encaja en esa línea. Es un prototipo de asistencia robótica construido sobre plataformas de bajo costo y software abierto, dentro de las limitaciones presupuestales de un entorno universitario.

La presentación del propio documento sigue la norma NTC 1486 del ICONTEC [57], que regula cómo se presentan las tesis y los trabajos de grado y que la plantilla institucional declara como su base.

2.4.1 Alcance de la operación y responsabilidad

El alcance del proyecto excluye de forma explícita que los robots interactúen mecánicamente con el edificio. No accionan puertas ni botones de ascensor, no transportan carga y no cruzan las zonas de transición vertical. Esa delimitación se adoptó por razones técnicas y de costo, y tiene además una consecuencia normativa favorable: evita intervenir sistemas sujetos a regulación específica de seguridad en edificaciones.

Las pruebas se hicieron en pasillos de uso común, en franjas de baja circulación, con una persona junto a cada vehículo durante las maniobras y la posibilidad de cancelar la misión desde la interfaz en cualquier momento.

2.5 Marco ambiental

El impacto ambiental del prototipo está en el hardware que emplea, la energía que consume y el destino de sus baterías. Las dos plataformas robóticas son equipo institucional que ya existía: el proyecto no las compró, y el presupuesto del anteproyecto las contabiliza como uso y desgaste. La decisión fue económica, pero también alarga la vida útil de equipos que estaban subutilizados. El entorno de evaluación es el propio edificio de la universidad, sin infraestructura dedicada ni modificaciones.

Cada vehículo lleva una fuente de alimentación para el cómputo y otra para la tracción, y una sesión de campo completa gasta una fracción de su carga. El mayor consumo del proyecto está en la estación de trabajo que ejecuta la simulación, con dos instancias del simulador a la vez y de forma sostenida. Eso respalda que la lógica de coordinación se verificara como función pura, sin simulador. Las baterías de polímero de litio de la tracción son el componente más sensible, porque contienen litio y se degradan con los ciclos de carga. Al final de su vida útil deben ir a los canales de disposición de residuos electrónicos de la institución, nunca a la basura doméstica. Como la tracción funciona en lazo abierto, una misma orden produjo velocidades distintas en corridas consecutivas, con la batería como causa más probable, y eso obliga a revisar su nivel de carga entre mediciones.

3. DESARROLLO

El sistema se construyó en dos semestres de 2026, con las seis fases que fijó el anteproyecto: formulación, levantamiento de requisitos, diseño conceptual, desarrollo progresivo, integración y pruebas, y validación con análisis de resultados. El primer semestre, de febrero a junio, cerró las tres primeras y empezó el desarrollo. El segundo, de agosto a octubre, hizo la integración y la evaluación, con un cronograma replanificado que aprobó el director (tabla 3.1). En esa segunda mitad, la matriz de requisitos asignó a cada requisito la prueba que lo declara cumplido, y los criterios de la demostración física fijaron qué debían superar los vehículos reales y hasta cuándo se tomaban datos (sección 3.5.4).

Tabla 3.1. Etapas del trabajo frente a las fases del anteproyecto.

Período	**Fase**	**Lo que se hizo**
Febrero y marzo	Planteamiento y requisitos	Anteproyecto, con el problema, la pregunta, la revisión de literatura y su sustentación; requisitos funcionales y restricciones técnicas
Abril	Requisitos y diseño conceptual	Análisis del DonkeyCar y de los simuladores; adopción del modelo del AWS DeepRacer y migración a ROS 2 Humble; arquitectura con servidor central y escenario de relevo
Mayo y junio	Desarrollo progresivo	Pasillo de la universidad en Gazebo; seguidor de paredes, descartado; sensores en el primer piso; primer mapa con SLAM
Agosto	Integración	En simulación: Nav2 para dirección Ackermann, contrato de interfaces, dos agentes aislados, entorno de dos pisos, requisitos, protocolo y primera misión con relevo. Con los vehículos: software de la comunidad, láser y mando desde ROS 2
Septiembre	Integración y validación	En simulación: campaña de treinta misiones, interfaz en un teléfono, confirmación de cambio de piso, cancelación y código congelado. Con los vehículos: segundo vehículo, banda muerta, decisión de la demostración física y primera navegación autónoma
Octubre	Validación	Pisos 3 y 4 levantados con flexómetro; odometría verificada; unidad inercial; llegada verificada; media vuelta; coordinador en el portátil; campaña con los dos vehículos

El capítulo empieza por el recorrido, porque varias decisiones solo se entienden en el orden en que se tomaron. Las secciones siguientes, una por objetivo específico, describen lo que quedó y por qué.

3.1 Recorrido del trabajo

Esta sección cuenta el trabajo en el orden en que se hizo. Lo que quedó de cada etapa se describe en las secciones de cada objetivo, a las que remite el texto.

3.1.1 La plataforma DonkeyCar y su simulador

El anteproyecto planteó dos vehículos tipo DonkeyCar con una Raspberry Pi 4, simulación en CoppeliaSim y visión con OpenCV. DonkeyCar es una plataforma abierta de vehículos a escala con dirección tipo automóvil, orientada a la conducción autónoma con una cámara [58]. En abril se instaló su simulador y se condujo desde la interfaz web de la plataforma. En paralelo se empezó un modelo propio del vehículo para ROS 2, con el chasis, las ruedas, la dirección y su árbol de transformaciones (figura 3.1).

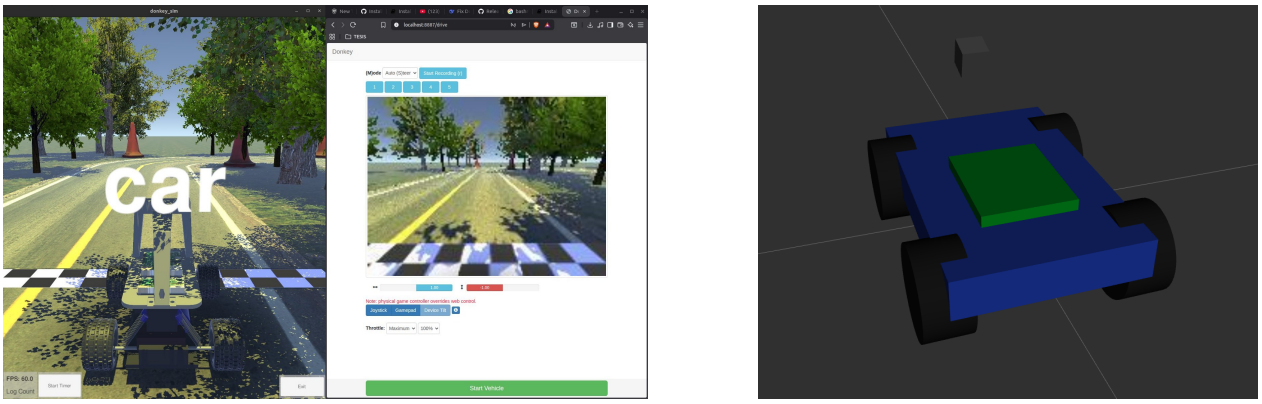


Figura 3.1. Exploración de la plataforma DonkeyCar. A la izquierda, su simulador conducido desde la interfaz web de la plataforma, con la imagen de la cámara del vehículo. A la derecha, el modelo propio del vehículo en el visualizador de ROS 2, con el chasis, las ruedas y la dirección.

Ese trabajo mostró tres limitaciones. El ecosistema DonkeyCar no se actualiza desde hace años y no es compatible con las versiones recientes de ROS 2. Su simulador no se comunica con ROS 2, así que no permite probar el mapeo, la localización, la planificación ni la comunicación entre robots, que son el núcleo del proyecto. Y no existía un modelo del DonkeyCar para Gazebo con dirección Ackermann que funcionara en ROS 2 Humble.

Para el simulador fijamos cuatro criterios: compatibilidad con ROS 2 Humble, vehículos con dirección Ackermann, entornos interiores y soporte para mapeo y navegación. Entre CoppeliaSim, que proponía el anteproyecto, el simulador del curso F1TENTH [59] y Gazebo [49], elegimos Gazebo por su integración directa con ROS 2 y con su sistema de control de articulaciones. También decidimos conservar un chasis con dirección tipo automóvil. Cada robot atiende un piso y la persona

cambia de nivel por la escalera, así que el vehículo no tiene que entrar a un ascensor ni subir escalones. La vuelta dentro del pasillo puede quedar a cargo de un planificador para automóviles.

3.1.2 El modelo de AWS DeepRacer y su migración a ROS 2 Humble

Una semana después cambiamos al modelo del AWS DeepRacer, y a nuestro juicio fue la decisión que hizo viable el proyecto en el tiempo disponible. Es la misma clase de plataforma que el DonkeyCar, con chasis a escala 1:18, dirección Ackermann, computador a bordo y software abierto. AWS publicó en código abierto su modelo de simulación, con un complemento de cinemática Ackermann para Gazebo y los nodos que traducen las órdenes de velocidad a los servos [60], y la universidad ya tenía dos unidades. Lo que pide el segundo objetivo, que es cinemática tipo automóvil, bajo costo y código modificable, se cumple igual con las dos plataformas, y usar las que existían dejó el tiempo para la coordinación. El costo fue convivir con el software del fabricante, cuyos nodos ocupan el láser, se ejecutan con el usuario administrador y publican sin espacio de nombres (sección 3.3.7).

El software de AWS estaba escrito para ROS 2 Foxy sobre Ubuntu 20.04, y el proyecto trabaja con Humble sobre Ubuntu 22.04. La migración necesitó tres cambios, todos en el sistema de control de articulaciones: la orden que activa los controladores pasó de `start` a `active`, y el componente que publica el estado de las articulaciones cambió de nombre y de tipo. En un mundo vacío (figura 3.2, izquierda), el vehículo apareció con el láser y la cámara publicando a 62 cuadros por segundo, con un factor de tiempo real de 1,00 y los siete controladores activos. La odometría registró 0,40 m/s ante una orden de 0,5 m/s.

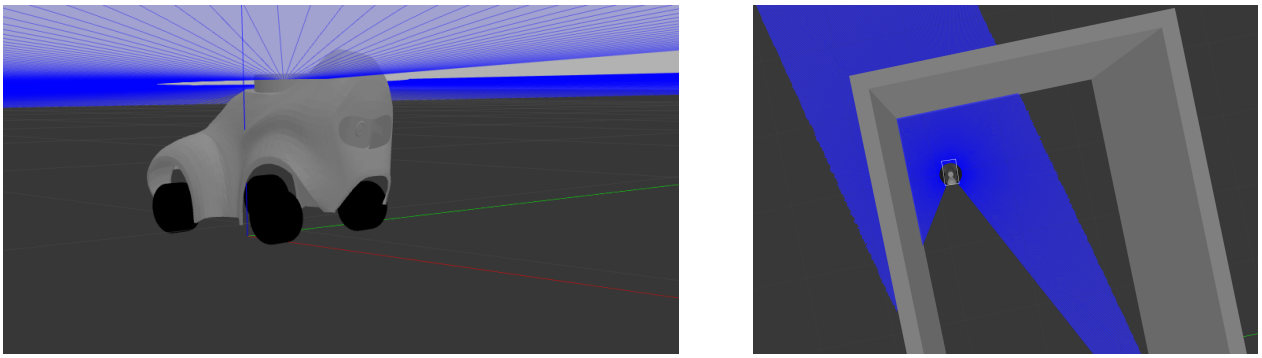


Figura 3.2. El modelo del DeepRacer migrado a ROS 2 Humble. A la izquierda, en un mundo vacío, con el barrido del láser en azul. A la derecha, dentro del primer modelo del pasillo de la universidad, de 7 por 3 m; el sector sin barrido detrás del vehículo es el que no cubría el láser del modelo simulado.

3.1.3 Los primeros entornos simulados y el seguidor de paredes

En mayo se modeló un tramo del pasillo de la universidad, de 7 por 3 m (figura 3.2, derecha). Sobre él se programó un seguidor de paredes, que el director sugirió como ejercicio de integración entre el láser y la cinemática del vehículo. La primera versión aplicaba control proporcional-derivativo sobre un rayo lateral del láser, y saturaba el giro porque no conocía la orientación del vehículo respecto a la pared. La segunda usó la técnica de dos rayos del curso F1TENTH, que estima ese ángulo y la distancia a la pared un metro más adelante.

En el pasillo de 7 por 3 m ninguna sintonía estabilizó al vehículo: el giro se saturaba y el vehículo trazaba círculos o chocaba, porque su radio de giro deja muy poco margen en 3 m de ancho. En un pasillo ampliado de 15 por 5 m convergió por tramos, a unos 0,76 m de la pared frente a un objetivo de 0,5 m, y se perdía en los extremos. La conclusión nos pareció clara: un comportamiento reactivo no anticipa las restricciones de un vehículo tipo automóvil en un pasillo estrecho. Por eso el proyecto pasó a navegar con un mapa y con un planificador que conoce esas restricciones.

Después se modeló la primera planta del edificio de pruebas, un pasillo de unos 44 por 5 m, y en ella se comprobaron el láser, la cámara, la odometría y el árbol de transformaciones. Con ese mundo se construyó el primer mapa por localización y mapeo simultáneos (SLAM), que se aceptó sin medirlo. Medido en septiembre, resultó de 23,04 por 15,72 m, la mitad del largo y el triple del ancho del pasillo. Fue la primera manifestación, sin diagnóstico entonces, de la falta de información de avance en un pasillo recto (sección 2.3.4), que acabó determinando cómo se hicieron los mapas (sección 3.3.4).

3.1.4 La integración en simulación

El segundo semestre empezó con un cronograma replanificado, aprobado por el director, con un criterio de cierre por semana. Lo primero fue ordenar la base. El espacio de trabajo donde se compilaba el código era una copia del repositorio del fabricante y no un enlace al del proyecto, así que los ajustes del mapeo del primer semestre nunca se habían ejecutado. Se enlazó al repositorio del proyecto y se fijó el contrato de interfaces (sección 3.2.2).

En agosto se aislaron los dos agentes, se construyó el entorno de dos pisos y se escribieron los requisitos y el protocolo. A finales de mes se completó la primera misión con relevo, y la campaña de treinta misiones corrió a comienzos de septiembre. Las dos decisiones de diseño de esa etapa, un simulador por vehículo y un mapa calculado en lugar del de SLAM, están en la sección 3.2.5.

También se corrigió el modelo del vehículo, que tomaba la velocidad angular de la orden estándar

de ROS 2 como el ángulo del volante. Ahora el ángulo se calcula con el modelo de bicicleta, a partir de las velocidades angular y lineal y de la distancia entre ejes, y el error de seguimiento de las trayectorias bajó a la mitad. La misma corrección se aplicó al nodo que mueve el vehículo real.

3.1.5 Del vehículo físico a la campaña con los dos vehículos

Con los vehículos se trabajó en paralelo desde agosto. Se instaló el software de la comunidad del DeepRacer [61] en una Raspberry Pi 4 y en la tarjeta original, y los vehículos quedaron con su tarjeta original, con Ubuntu 24.04 y ROS 2 Jazzy (sección 3.3.1). Como la integración de Gazebo 11 con ROS 2 no está publicada para Jazzy, desde entonces el proyecto trabaja con dos distribuciones (sección 3.2.6).

El camino hasta la campaña tuvo tres giros, que se tratan con la plataforma. Una supuesta avería resultó ser un problema de permisos, y la tarjeta se saturó con dos cadenas de navegación (sección 3.3.7). Además, el modelo del piso 2 no correspondía al edificio (sección 3.3.4). El 21 de septiembre decidimos intentar la demostración física con seis pruebas de verificación (sección 3.5.4), y la campaña con los dos vehículos corrió el 16 de octubre.

3.2 Arquitectura funcional del sistema

Esta sección corresponde al primer objetivo específico: modelar la arquitectura funcional, con sus requisitos, sus escenarios de operación en varios pisos, la estrategia de asignación de tareas y el esquema de comunicación entre robots. La figura 3.3 muestra el sistema tal como se implementó en simulación: la interfaz por la que entra la solicitud, el coordinador que la reparte, los dos agentes con su pila de navegación y la escalera donde ocurre el relevo.

3.2.1 Requisitos

El anteproyecto no enumeró los requisitos del sistema, aunque su metodología pedía verificar la implementación contra ellos. Los reconstruimos al iniciar la integración a partir de los objetivos específicos, el alcance y el contrato de interfaces. Cada requisito cita su fuente y tiene una prueba que lo declara cumplido o no, y un requisito sin prueba no entra en la matriz.

La matriz tiene treinta y seis requisitos: veintinueve describen funciones del sistema y siete son restricciones del alcance, como operar en interiores y en condiciones controladas o correr la



Figura 3.3. Arquitectura del sistema en simulación, con un proceso de Gazebo por piso. Ningún robot atraviesa la zona de transición: lo que la cruza es el usuario, y lo que se traspasa entre agentes es la responsabilidad de guiarlo.

simulación en tiempo real. Al primer objetivo le corresponden diez, sobre arquitectura, coordinación y relevo; al segundo, seis, sobre la plataforma; al tercero, seis, sobre la interfaz; y al cuarto, siete, sobre las métricas, el registro de cada misión y las dos campañas. La confirmación de cambio de piso y la cancelación no estaban en el anteproyecto: las pidió el director al ver el sistema en uso. El avance de cada objetivo es el promedio del estado de sus requisitos.

3.2.2 El contrato de interfaces

Antes de escribir código se fijó un contrato: qué habla con qué y en qué formato. Un coordinador central decide y manda, dos agentes, uno por nivel, ejecutan, y una interfaz web es la vía por la que el usuario pide el servicio. La comunicación sigue tres reglas:

1. El coordinador manda a un agente solo mediante la acción de navegación de ROS 2; no publica velocidades directamente. En la versión final puede pedirle además una media vuelta, que el agente ejecuta por su cuenta antes de navegar (sección 3.3.8).
2. La interfaz habla solo con el coordinador, nunca con los agentes.
3. El texto que ve el usuario lo redacta el coordinador, y la interfaz lo muestra literal.

La tercera regla nos parece la más valiosa del contrato. Si la página interpretara el estado para decidir qué mostrar, habría dos copias de la misma lógica, y con el tiempo acabarían contradiciéndose.

El contrato se implementó como un paquete propio de mensajes de ROS 2. La acción de guiado recibe el origen y el destino, devuelve el estado durante la misión y, al final, el tiempo total y el número de relevos o el motivo del fallo. Cada cambio de etapa (tabla 3.2) se publica, y esas publicaciones son las marcas de tiempo con que se calculan las métricas.

3.2.3 El planificador, escrito sin ROS

El relevo es el aporte declarado del proyecto, y por eso escribimos su planificación como una función pura, sin dependencia de ROS 2 ni estado propio. Entran el catálogo de puntos y un par origen-destino, y salen los tramos que hay que ejecutar. Si el origen y el destino están en el mismo nivel, la misión tiene dos tramos del mismo agente: ir a recoger al usuario y llevarlo al destino. Si están en niveles distintos, tiene cuatro: los dos anteriores, terminando en el punto de transferencia del nivel de origen; el desplazamiento del segundo agente a su punto de transferencia; y el guiado

Tabla 3.2. Etapas de una misión, tal como las publica el coordinador.

Etapa	Significado
Recibida	El coordinador aceptó la solicitud y todavía no asignó agente
Primer tramo	El agente del nivel de origen va a recoger al usuario y lo guía
Transferencia	El agente del otro nivel va a su punto de transferencia
Esperando confirmación	El sistema espera que el usuario confirme que ya cambió de piso
Segundo tramo	El agente del nivel de destino guía al usuario hasta el final
Completada	La misión terminó con la llegada verificada
Fallida	La misión terminó sin cumplirse, con el motivo en el texto
Cancelando	El usuario canceló y el agente activo regresa a su punto de transferencia
Inactiva	No hay misión en curso

final. El punto de encuentro no se negocia: es el punto de transferencia declarado de cada nivel en el catálogo.

A nuestro juicio fue una de las decisiones más rentables del proyecto. Con el relevo dentro del nodo, mezclado con los clientes de acción, la única forma de comprobarlo habría sido levantar dos simuladores y observar si el vehículo llega. Como función pura, una prueba recorre las 930 combinaciones de origen y destino del catálogo en milisegundos, sin simulador. Esa prueba encontró un defecto que una corrida no habría delatado: cuando el origen o el destino era el propio punto de transferencia de su nivel, el plan mandaba al mismo robot dos veces a la misma pose. Nav2 devuelve éxito al instante ante un desplazamiento de cero metros, así que en simulación no se habría visto.

3.2.4 La asignación de tareas

Cada nivel tiene un agente asignado, y esa correspondencia es un dato de configuración, no una constante del código. El coordinador elige el agente al aceptar la solicitud, según el nivel del origen y del destino. Con un solo agente admisible por nivel no hace falta un optimizador, porque el método húngaro devolvería la misma tabla (sección 2.3.1).

El planificador exige un único punto de transferencia por nivel y falla con un mensaje explícito si encuentra dos. Hoy cada planta tiene una escalera y eso se cumple; la verificación está para que, el día que haya dos, la decisión se tome a la vista. En los vehículos, además, el coordinador no da por buena la llegada que informa Nav2, sino que la comprueba con la pose que publica el agente (sección 3.5.3).

3.2.5 Escenarios de operación en simulación

Los escenarios de varios pisos se construyeron primero en Gazebo 11 con ROS 2 Humble. La integración de Gazebo con ROS 2 en Humble aplica a todos los complementos el espacio de nombres del primer modelo cargado. Por eso no fue posible tener dos robots en un mismo proceso, y cada vehículo tiene su propio simulador. El catálogo llegó a 31 puntos de interés: quince en el piso 1 y dieciséis en el piso 2.

Al principio los dos pisos estaban en un mismo mundo, y el láser simulado, de 10 m de alcance, veía paredes del otro piso en 61 de 1701 posiciones muestreadas. Esas posiciones estaban justo donde arranca el primer vehículo y donde está su punto de transferencia. Por eso cada piso es ahora un mundo distinto, con su propio proceso de simulación. Los dos vehículos comparten un dominio de comunicaciones, separados por espacio de nombres (sección 2.3.5), y la figura 3.4 muestra una misión con relevo entre los dos procesos.

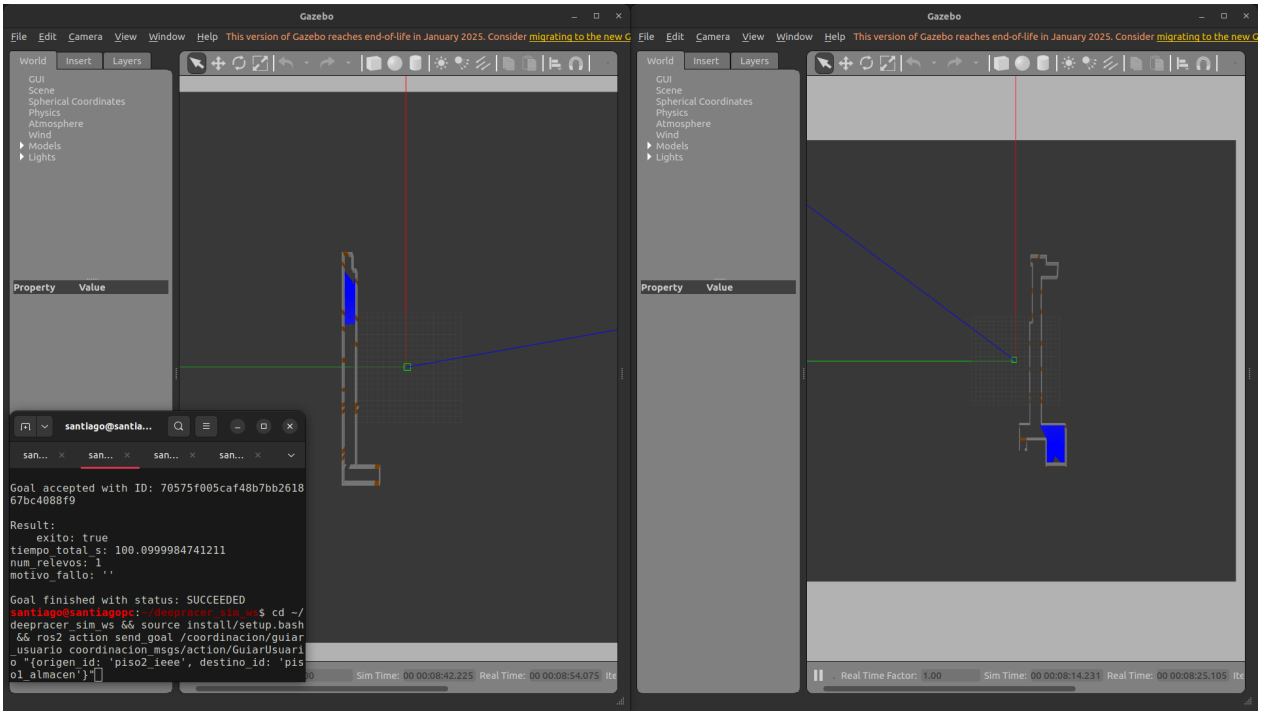


Figura 3.4. Misión con relevo en simulación. Cada ventana es un proceso de Gazebo con un piso y su vehículo; el láser de cada uno se ve en azul. La terminal muestra el resultado de la acción: éxito, un relevo y 100,1 s de duración.

El mapa de navegación tampoco salió de SLAM. El que se había construido cubría el 35,5 % del largo del pasillo, medía 15,0 m de ancho frente a 5,3 m reales, y 2688 de sus 3618 celdas ocupadas, el 74 %, no tenían ninguna pared detrás. Lo reemplazamos por un mapa calculado de la geometría del modelo, exacto en simulación; como un mapa de ocupación es un corte horizontal, el del piso

superior resultó idéntico. Por último, la simulación debe correr a tiempo real para que las métricas temporales valgan, y en las treinta misiones de la campaña el factor de tiempo real tuvo un mínimo de 0,9917 y una media de 0,9973.

3.2.6 De la simulación a los vehículos

Al llevar el sistema a los vehículos se mantuvo su estructura: un coordinador, un agente por piso, el relevo en la escalera y las reglas del contrato (sección 3.2.2). También se mantuvo el código. El del coordinador, el del planificador, el de los mensajes y el de la página es el mismo en los dos destinos: compila sin cambios en las dos distribuciones de ROS 2 y completó una misión en cada uno. Lo que cambió fue la implementación de los bloques. La tabla 3.3 reúne cada cambio con su causa, y la figura 3.5 muestra el sistema como funcionó en la campaña con los dos vehículos.

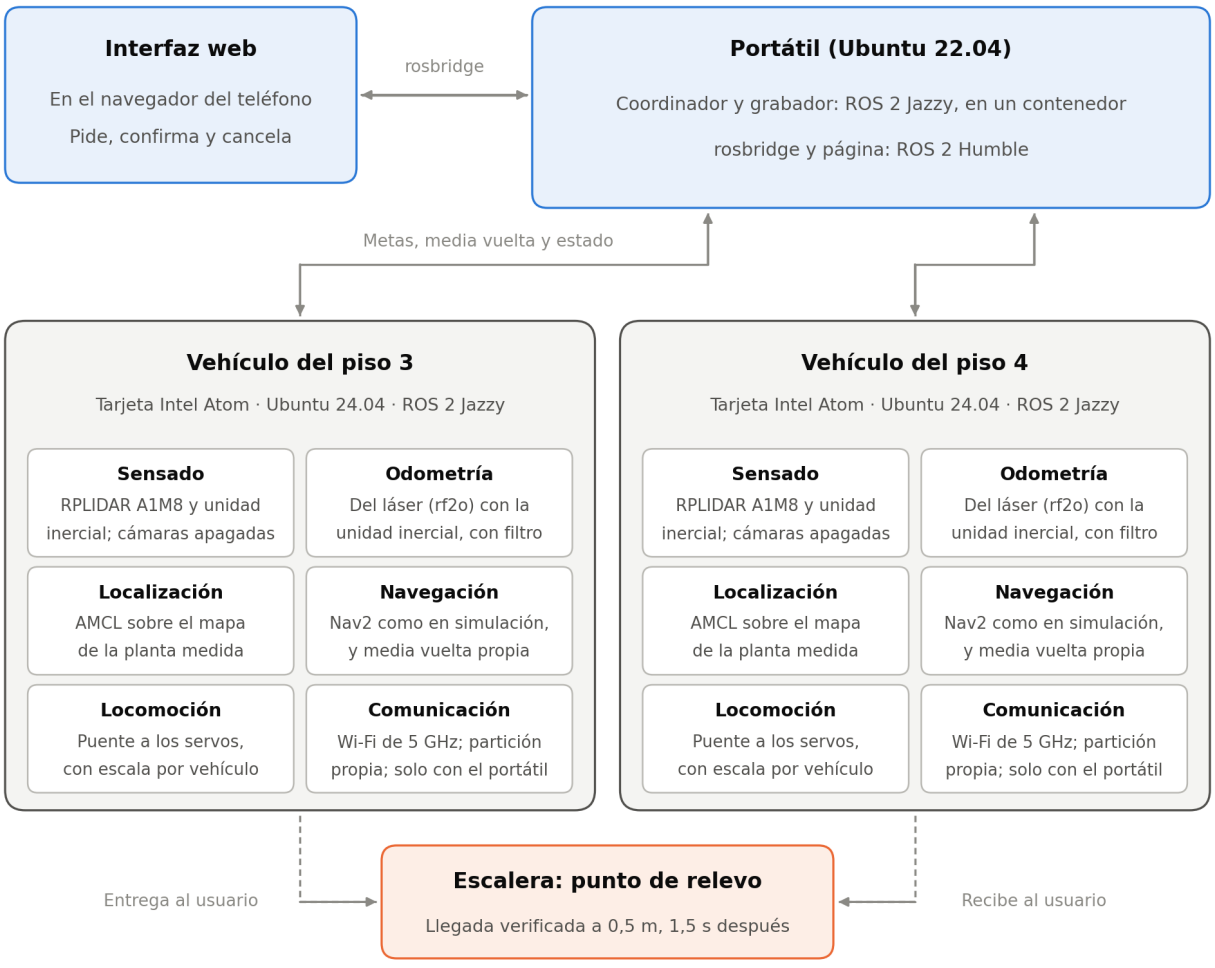


Figura 3.5. Arquitectura del sistema en la campaña con los dos vehículos, en los pisos 3 y 4. La estructura es la de la figura 3.3; cambia la implementación de los bloques.

Tabla 3.3. Cambios en la implementación al pasar de la simulación a los vehículos.

Parte	En simulación	En los vehículos	Causa
Distribución de ROS 2	Humble en todos los componentes	Jazzy en los vehículos y en el coordinador, que corre en un contenedor del portátil; Humble en la página y en rosbridge	El software de la tarjeta original solo existe para Ubuntu 24.04 con Jazzy, la simulación no existe en Jazzy, y la acción de navegación de Jazzy no es compatible con la de Humble (sección 2.3.2)
Coordinador	En el portátil, junto a los dos simuladores	En el portátil, después de un intento en uno de los vehículos	Con el coordinador y el grabador en una tarjeta y las dos navegaciones encendidas, la carga subió de 6 a entre 17 y 33 y Nav2 no arrancaba (sección 3.3.7)
Comunicación	Un solo computador y un dominio de comunicaciones; un espacio de nombres por agente	Red inalámbrica con un punto de acceso de 5 GHz por piso; una partición por vehículo y topología en estrella hacia el portátil	Los tópicos del fabricante no llevan espacio de nombres, y unos cuarenta procesos anunciándose por la red saturaban las tarjetas (sección 3.3.7)
Locomoción	Complemento Ackermann de Gazebo con `ros2_control`	Puente de la orden de velocidad a los servos, con una escala por vehículo	La tracción real tiene banda muerta y funciona en lazo abierto (sección 3.3.5)
Sensado	Láser simulado de 10 m de alcance	RPLIDAR A1M8 y la unidad inercial de la tarjeta; cámaras apagadas	Las cámaras no se usan y cargaban la tarjeta (sección 3.3.1)
Odometría	La del simulador	La del láser, combinada con la unidad inercial mediante un filtro	El vehículo no tiene codificadores en las ruedas, y con el láser solo el rumbo derivaba 5,9° por minuto (secciones 3.3.3 y 3.3.6)
Mapa	Calculado de la geometría del mundo simulado	Derivado de la planta medida con flexómetro	El mapeo con SLAM a bordo no dio un mapa utilizable en ningún pasillo (sección 3.3.4)
Escenario	Pisos 1 y 2 del modelo del edificio	Pisos 3 y 4, levantados con flexómetro	El modelo del piso 2 no correspondía al edificio, y en sus pasillos uniformes la odometría del láser no veía el avance (sección 3.3.4)
Llegada	Tolerancia de 0,25 m	Tolerancia de 0,5 m, verificada con la pose del agente 1,5 s después de que Nav2 da la meta por alcanzada	Por la banda muerta, el vehículo no alcanza la precisión de la simulación, y sigue rodando cuando Nav2 declara la llegada (secciones 3.3.5 y 3.5.3)
Media vuelta	La planifica Nav2, con marcha atrás	Maniobra propia del agente, vigilada con el láser y el mapa	En la salida del piso 4, de 2,5 m de ancho, la media vuelta de Nav2 abortó contra la pared (sección 3.3.8)

3.3 Plataforma robótica

Esta sección corresponde al segundo objetivo específico: desarrollar una plataforma de dos vehículos con los módulos de locomoción, sensado, procesamiento y comunicación que necesita la operación colaborativa. La elección del AWS DeepRacer en lugar del DonkeyCar se justifica en la sección 3.1.2.

3.3.1 Componentes del vehículo

Los dos vehículos operan con su tarjeta original, para la que el software de la comunidad solo se publica sobre Ubuntu 24.04 con ROS 2 Jazzy. La tabla 3.4 reúne los componentes. Cada vehículo atiende un piso, uno el 3 y otro el 4. El láser que traía el kit no era el que describía el primer informe de hardware; se identificó leyendo el modelo que reporta el propio sensor.

Tabla 3.4. Componentes de cada vehículo y del computador central.

Componente	Descripción
Chasis	AWS DeepRacer, escala 1:18, dirección Ackermann, motor de tracción con variador y servomotor de dirección
Cómputo	Tarjeta original, procesador Intel Atom E3930 de dos núcleos, 4 GB de memoria; Ubuntu 24.04 y ROS 2 Jazzy
Láser	RPLIDAR A1M8-R5: 360°, 360 muestras por barrido, alcance de 0,15 a 12 m, entre 6,8 y 7 Hz
Unidad inercial	Bosch BMI160 integrada en la tarjeta, por I2C: giroscopio y acelerómetro
Cámaras	Estéreo; desactivadas por software, porque el sistema no las usa y cargaban la tarjeta
Alimentación	Una batería para el cómputo y otra de polímero de litio para la tracción
Computador central	Portátil con Ubuntu 22.04 y ROS 2 Humble (AMD Ryzen 5 PRO 5650U, 16 GB); el coordinador corre en un contenedor con Jazzy

3.3.2 Nav2 sobre un vehículo que no gira sobre su eje

La configuración por omisión de Nav2 asume tracción diferencial y propone trayectorias que un vehículo tipo automóvil no puede ejecutar. Por eso sustituimos dos de sus tres capas (sección 2.3.3). La planificación global pasó al planificador híbrido A* de Smac con modelo Reeds-Shepp, y el seguimiento, al controlador de persecución pura regulada, los dos con marcha atrás, y con una velocidad deseada de 0,5 m/s. Del árbol de comportamiento se quitó la recuperación que gira sobre

el propio eje, con la que el vehículo, ante un bloqueo, movía las ruedas sin avanzar hasta agotar los reintentos. Los parámetros tienen una variante por distribución, y una prueba automática falla si se separan en algo distinto de las diferencias documentadas.

La cadena se puso en marcha con la escalera de siete pasos de la guía oficial de Nav2 [62]. El orden va de las transformaciones entre marcos de referencia al planificador y el controlador, pasando por la odometría, el láser, el mapa, la localización y los mapas de costos. Seguir ese orden permitió aislar el problema del apartado siguiente, porque una odometría que no mide y un planificador que no planifica se manifiestan igual: el vehículo no llega. El orden importa también al arrancar, porque los mapas de costos leen el mapa al configurarse. Por eso el programa de arranque de los vehículos activa primero el servidor de mapas y espera a que los seis nodos de la cadena estén activos. En el laboratorio, la cadena quedó lista en 4 min 4 s en un vehículo y en 4 min 22 s en el otro.

3.3.3 La odometría del láser como limitación principal

El vehículo no tiene codificadores en las ruedas, y su única odometría sale de comparar barridos sucesivos del láser. Durante meses no dio una medida utilizable, por falta de información en la señal (sección 2.3.4): en un pasillo recto y uniforme, cada barrido es casi idéntico al anterior y el estimador devuelve un avance cercano a cero. En los pasillos de los pisos 1 y 2 la odometría registraba entre el 1,3 % y el 73 % del avance real, y la localización llegaba a separarse más de dos metros de la posición medida con flexómetro (figura 3.6).

Ningún ajuste del algoritmo recupera información que la señal no contiene, y por eso la solución fue cambiar de entorno. El criterio salió de la misma medida: hace falta estructura no uniforme dentro del alcance del sensor durante todo el recorrido. Los pisos 3 y 4 la tienen. Entre las hornacinas de los salones, los casilleros, la papelera, el ascensor y la escalera suman ocho cambios de perfil en 25 m, uno cada tres metros. Con la misma odometría y sin cambiar un parámetro, el error cayó por debajo del 5 % (tabla 4.3), y el director aceptó el cambio de sitio. Consideramos que fue la decisión que hizo posible la demostración física, porque en los pasillos de los pisos 1 y 2 ningún ajuste habría dado al vehículo la información que le faltaba.

3.3.4 Los mapas, derivados de medidas y no de SLAM

Durante semanas intentamos construir el mapa con SLAM a bordo de los vehículos, y no se consiguió en ningún pasillo, por la razón del apartado anterior. La vía que funcionó fue levantar la planta con flexómetro y derivar de ese levantamiento tanto el mundo simulado como el mapa de navegación.

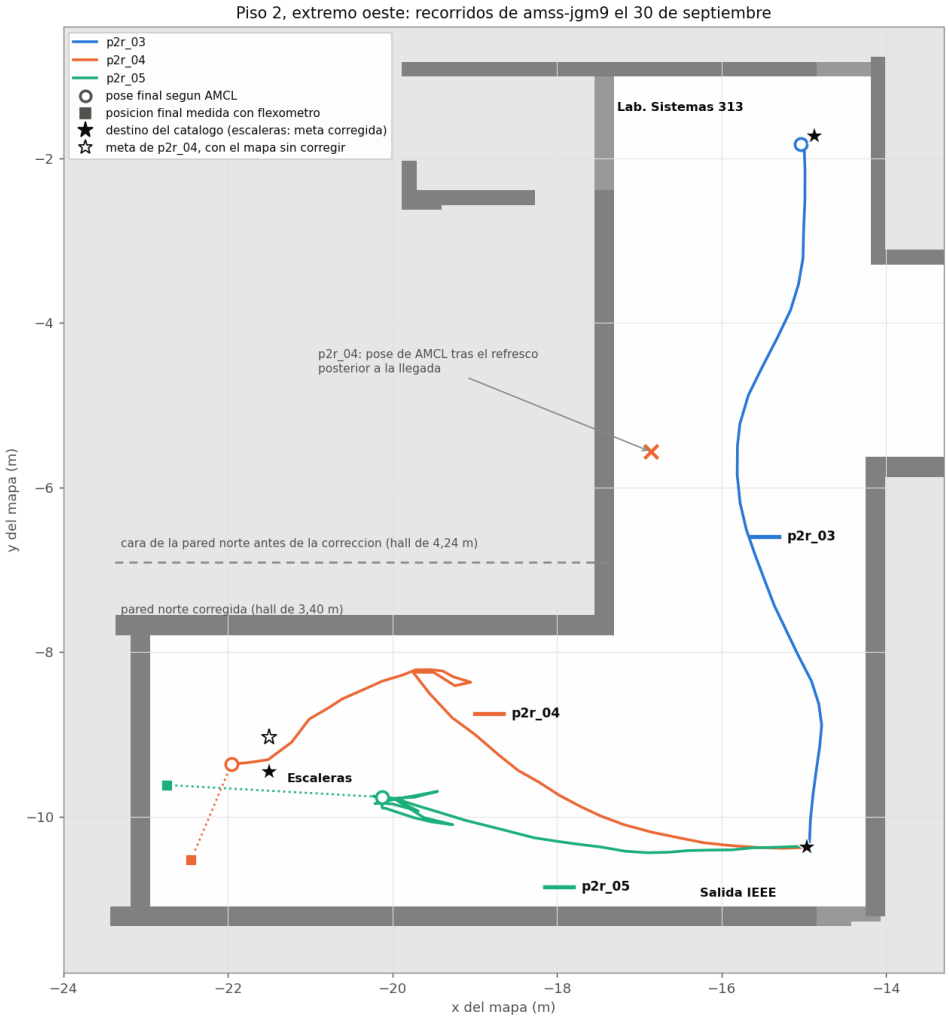


Figura 3.6. Tres recorridos en el hall del piso 2. El círculo es la pose que reporta la localización al terminar; el cuadrado, la posición real medida con flexómetro. La separación entre los dos es el error, y su causa está en la odometría, no en el algoritmo de localización.

Así, simulación y edificio coinciden porque salen de la misma fuente. Esa correspondencia se había dado por supuesta, y al verificarla por primera vez resultó falsa. El modelo anterior daba el hall del piso 2 0,84 m más ancho de lo que es, 4,24 m frente a 3,40 m, un 25 % de error.

La herramienta que genera las plantas suma las dos paredes de cada pasillo y avisa si no cierran dentro del 2 %. Esa comprobación encontró cinco errores antes de generar nada, y las dos plantas cierran al 0,32 % y al 0,62 %. La figura 3.7 es el mapa del piso 4, con sus cinco puntos de interés: tres salones, el ascensor y la escalera, que es su punto de transferencia.

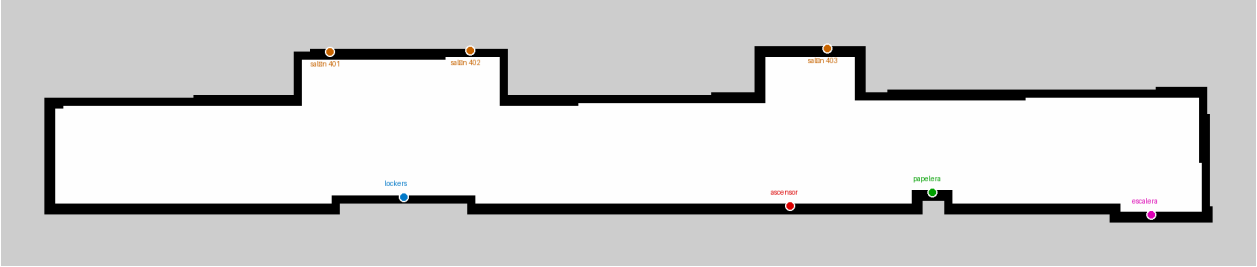


Figura 3.7. Mapa de navegación del piso 4, derivado del levantamiento con flexómetro. El pasillo corre 25 m del extremo norte al sur; los salones quedan al este, y el ascensor y la escalera al oeste.

3.3.5 La banda muerta de tracción

Al integrar la cadena de tracción apareció una característica de la plataforma que condiciona la llegada: el vehículo no se mueve con velocidades pedidas por debajo de 0,40 m/s. El puente que traduce la orden al variador las convertía en tracción nula, sin ningún aviso, y el vehículo se detenía al entrar en la fase final del recorrido, intentando moverse a una velocidad que su propia cadena anulaba. Se subió a 0,40 m/s la velocidad mínima de aproximación del controlador, y después se modificó el puente para que envíe las órdenes menores con el escalón de tracción más bajo.

El resultado es que el vehículo se aproxima a la meta a 0,40 m/s y no tiene régimen de aproximación fina. Es un límite de la plataforma, que no se corrige afinando el controlador. Como la tracción funciona en lazo abierto, la relación entre la orden y la velocidad real tampoco es igual en los dos vehículos. Por eso el puente tiene una escala por vehículo, ajustada en campo: 0,85 en el del piso 3 y 0,9 en el del piso 4. Con la escala 1,0, el del piso 4 iba a 1,36 m/s y se pasaba de la meta; con 0,9 va a unos 0,7 m/s.

Ante ese límite, la tolerancia de llegada con los vehículos se fijó en 0,5 m, por decisión del director, mientras la simulación conserva 0,25 m. Bajar el umbral de arranque del vehículo queda para los trabajos futuros.

3.3.6 La unidad de medición inercial de la tarjeta

Al principio el rumbo también salía de la odometría del láser, que en un pasillo recto pierde los giros igual que el avance. Con el vehículo quieto, ese rumbo derivó hasta 5,9° por minuto, y un giro de 90° hecho a mano se registró como 19,1°. La tarjeta incluye una unidad de medición inercial, una Bosch BMI160 que el software del fabricante no usa. Leída directamente, midió giros de 90° y 360° con un error de 0,1 % a 1,3 %, y quieta no se desvió más de 0,3° en 60 s.

Escribimos un nodo que la lee y calibra el sesgo del giroscopio al arrancar. Un filtro de Kalman extendido (sección 2.3.4) toma de la odometría del láser el avance y de la unidad inercial el giro; la unidad publica a 25 Hz y el filtro corre a 15 Hz, para no cargar la tarjeta. En un giro de 90° contra una línea del piso, el filtro midió 90,17° en un vehículo y 88,38° en el otro, y con el vehículo quieto su rumbo se movió 0,3° o menos por minuto. La posición no mejora, porque sigue saliendo del láser. Con el rumbo estable se redujo de 1,0 a 0,5 m el margen en el que Nav2 da la meta por alcanzada, y con ese margen se superó la prueba de navegación de un vehículo (sección 4.2.2).

3.3.7 Los dos vehículos en la misma red

Con los dos vehículos encendidos aparecieron tres problemas que con uno solo no existían. El primero fue de permisos. El software del fabricante se ejecuta con el usuario administrador, y ROS 2 intercambia los datos entre procesos del mismo equipo por memoria compartida, a la que solo ese usuario puede escribir. Los programas lanzados con otro usuario veían los nodos del fabricante pero no podían intercambiar mensajes con ellos, sin ningún error, y durante tres semanas eso pareció una avería de uno de los vehículos.

El segundo fue de nombres. Los tópicos del fabricante no llevan espacio de nombres, así que una orden de movimiento llegaba a los dos vehículos y la odometría de cada uno recibía el láser del otro. Cada vehículo tiene ahora una partición de comunicaciones propia, y la cadena del proyecto corre con un espacio de nombres distinto en cada uno.

El tercero apareció en el primer intento de misión con relevo. Con las dos cadenas completas, unos cuarenta procesos se anunciaban unos a otros por la red inalámbrica. La carga de la tarjeta de un vehículo pasó de 6 a entre 17 y 33, con el procesador al 84–95 %, y Nav2 no llegaba a arrancar. La respuesta fue la topología en estrella del diseño (sección 2.3.2): los procesos de cada vehículo solo se buscan dentro de él, lo que el coordinador necesita solo se anuncia al portátil, y los vehículos no se ven entre sí. El coordinador y el grabador pasaron al portátil, en un contenedor con Jazzy, y verifican las llegadas con la pose que cada vehículo publica dos veces por segundo.

Con todo el sistema en el laboratorio, las tarjetas quedaron al 54 % y al 59 % de uso en reposo, y al 68 % y al 74 % navegando con las ruedas en el aire. El controlador de Nav2, que corre a 10 Hz, no bajó de 5,5 Hz. La red tiene un punto de acceso de 5 GHz por piso, en la misma subred. Entre los dos vehículos, el tiempo de ida y vuelta de los mensajes tuvo una mediana de 5,99 ms y un percentil 95 de 11,12 ms, sin pérdidas, frente al límite de 100 ms que fija el requisito.

3.3.8 La media vuelta en varios tiempos

Por acuerdo con el director, cada misión empieza donde terminó la anterior, y el siguiente usuario queda a menudo detrás del vehículo, que en pasillos de 2,5 a 3,2 m de ancho tiene que dar la vuelta en varios tiempos. En el piso 4, a la salida del pasillo de 2,5 m, la media vuelta de Nav2 abortó dos veces contra la pared, porque el planificador proponía cada segundo una maniobra distinta. Además, el motor retrocedía a unas 1,7 veces la velocidad de avance, y se añadió al puente una escala de 0,75 para la marcha atrás.

Preferimos escribir la media vuelta como una maniobra propia del agente antes que seguir ajustando Nav2, que en ese caso trabajaba en el borde de lo que su controlador resuelve (sección 2.3.3). La maniobra alterna marcha atrás y avance con la dirección a tope y empieza hacia el costado con más espacio. Cada tiempo se corta cuando la huella prevista del vehículo queda a menos de 0,10 m de un obstáculo, visto por el láser o marcado en el mapa, donde la escalera está cerrada porque el láser no ve su borde.

El coordinador la pide antes de una meta que queda a más de 110° del rumbo del vehículo y a más de 1,5 m, y si no se completa en 120 s, sigue Nav2. En sus primeras pruebas giró 160° y 167°, en un solo tiempo cada vez, y en la campaña con los dos vehículos se ejecutó 24 veces (sección 4.3).

3.4 Interfaz de interacción con el usuario

Esta sección corresponde al tercer objetivo específico: programar una interfaz móvil de interacción humano-robot basada en la selección del origen y el destino.

3.4.1 Una página web, no una aplicación

El objetivo pide «una interfaz móvil», y construimos una página web que se abre en el navegador del teléfono. A nuestro juicio cumple el objetivo mejor que una aplicación para el usuario que el

proyecto declara, porque un visitante ocasional no va a instalar nada para preguntar por un salón. La página funciona sin salida a internet, porque la red del edificio puede no tenerla, y no usa bibliotecas ni recursos externos. Son un archivo HTML, una hoja de estilo y dos programas en JavaScript, 658 líneas en total, que sirve el propio portátil. Se comunica con ROS 2 por el puente *rosbridge*, que no tiene autenticación, así que el cortafuegos del portátil solo admite conexiones desde la red de los vehículos.

El primer programa es un cliente propio del protocolo del puente, porque la biblioteca de cliente más difundida implementa las acciones de la primera generación de ROS (sección 2.3.6). El segundo es la lógica de la página. Recibe el catálogo de puntos al conectarse y el estado de la misión una vez por segundo, pide y cancela con la acción de guiado y publica la confirmación de cambio de piso.

3.4.2 Lo que el usuario ve

El usuario elige el origen en una lista y el destino en un catálogo con búsqueda por nombre, donde cada destino muestra su piso y, si queda en el otro nivel, la marca «con relevo». Durante la misión, la página muestra un título calculado a partir de la etapa, el texto que redacta el coordinador, el vehículo que está guiando, hacia dónde va, el destino final y la distancia restante. Una franja de cinco pasos (preparando, tramo 1, relevo, tramo 2 y fin) marca el paso actual (figura 3.8).

La interfaz se verificó contra el coordinador real: el catálogo, una misión con sus etapas, el texto del coordinador mostrado literal y el botón que se reactiva al terminar. También se operó una misión completa con relevo desde un teléfono real conectado a la red de los vehículos, sin tocar el computador. Las dos funciones que siguen se añadieron después de ver el sistema en uso, a pedido del director.

3.4.3 Confirmación de cambio de piso

En una misión entre niveles, el segundo vehículo llega a su escalera y espera al usuario, pero no puede saber si la persona ya subió. Sin una pausa explícita, la misión podía darse por completada con el usuario todavía en el piso equivocado, y la instrumentación lo habría registrado como éxito. Por eso, cuando el segundo vehículo llega a su punto de transferencia, el coordinador pasa a la etapa de espera de confirmación. La página muestra entonces un botón para confirmar el cambio de piso, que solo existe en esa etapa (figura 3.9). A los 60 s sin confirmación aparece un aviso, y a los 120 s la misión termina como fallida. Los plazos se cuentan desde que llega el vehículo, para que un vehículo lento no le consuma tiempo al usuario, y se miden con el reloj del sistema y no con el de la

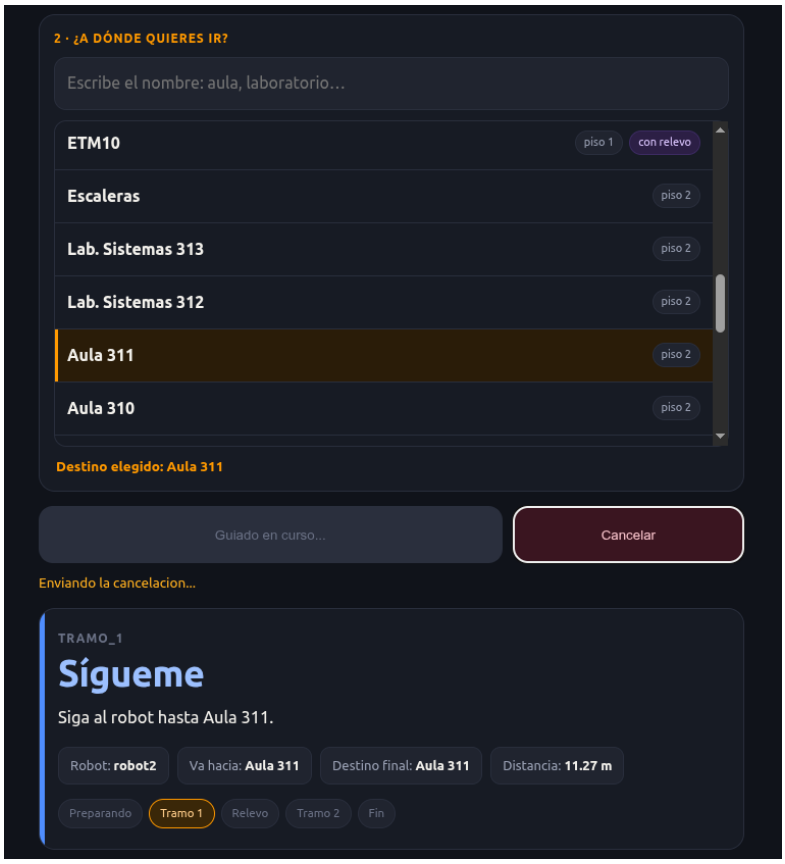


Figura 3.8. Interfaz durante una misión en simulación: el destino elegido, el botón de cancelar y el panel de estado del primer tramo, con el texto del coordinador, el vehículo activo y la distancia restante.

simulación. Si el usuario confirma antes de que el vehículo llegue, la confirmación se retiene y se aplica al llegar.

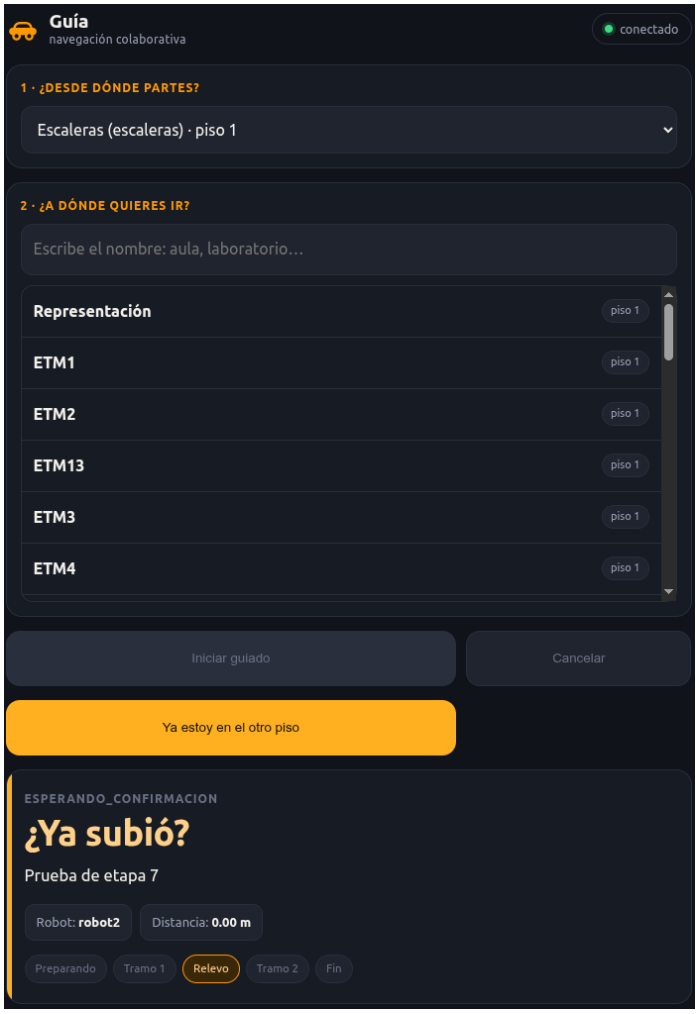


Figura 3.9. Interfaz en la etapa de espera de confirmación, durante su prueba automática: el estado de la misión lo publica la prueba, con el texto «Prueba de etapa 7», y la página muestra el botón «Ya estoy en el otro piso», de la versión anterior a los ajustes de la sección 3.4.5.

Se verificó en simulación con una corrida por camino. Confirmar a los pocos segundos termina la misión con éxito, y confirmar pasados los 60 s hace aparecer el aviso en el teléfono. No confirmar deja la misión fallida a los 120 s, con el motivo redactado.

3.4.4 Cancelación

El botón de cancelar existía pero no hacía nada, y no se había detectado. El servidor de acción del coordinador no registraba la función que responde a la cancelación, y ROS 2 rechaza por omisión, sin informar, toda cancelación que no tenga esa función. Al corregirlo se añadió el comportamiento

que faltaba: la misión pasa por la etapa de cancelación, el vehículo activo regresa al punto de transferencia de su piso, y la misión cierra como fallida con un texto que lo dice. En simulación, con una corrida cancelada y una de control, el vehículo quedó a 0,121 m de su punto de regreso.

Con los vehículos se cancelaron seis misiones, al menos una en cada etapa, y en las seis el vehículo volvió a su escalera, a entre 0,07 y 0,34 m de su punto. Esas pruebas mostraron un defecto que la simulación no podía mostrar, porque la media vuelta propia se añadió después. Nav2 conserva unos minutos las metas terminadas, y mientras tenía una cancelada en su lista, el agente marcaba el vehículo como en error y rechazaba la media vuelta. Sin ella, Nav2 intentaba girar por su cuenta en el pasillo y oscilaba. Ahora la media vuelta solo espera a que no haya una meta en curso: antes de la corrección se rechazaron ocho de quince medias vueltas, y después ninguna de diez.

3.4.5 Ajustes tras las pruebas con los vehículos

Las pruebas con los vehículos mostraron tres defectos de la página. El título de la etapa de confirmación preguntaba siempre «¿Ya subió?», lo que al subir resultaba brusco y al bajar del piso 4 al 3 era falso. Ahora sale de comparar el piso de origen con el del vehículo que espera, como «Suba con calma al piso 4» o «Baje con calma al piso 3». El botón dice «Ya estoy en el piso 4», o el piso que corresponda.

Ese cambio se hizo el día de la campaña, antes de correrla, y aun así los teléfonos mostraron la versión anterior, porque el servidor no le indicaba al navegador cuándo volver a pedir la página. Ahora la página se sirve con la indicación de comprobarla cada vez que se abre. El tercer defecto fue de nombres: los dos ascensores se llamaban igual en la lista y en una misión se eligió el del piso equivocado, así que los nombres repetidos se muestran ahora con su piso, como «Ascensor (piso 3)». Los tres ajustes se verificaron con pruebas automáticas y después con un teléfono y los vehículos, en una misión de subida y otra de bajada.

3.5 Protocolo experimental e instrumentación

Esta sección corresponde al cuarto objetivo específico: evaluar el sistema con el tiempo de respuesta, el tiempo de asignación, la tasa de éxito y la continuidad del servicio entre niveles. Escribimos el protocolo antes de instrumentar el sistema, y lo hicimos a propósito: un protocolo redactado después de ver los datos tiende a acomodarse a lo que salió, por ejemplo eligiendo el umbral que deja fuera las corridas malas. El protocolo fija las métricas con su definición operativa, el criterio de éxito, el tamaño de muestra y una lista cerrada de causas de descarte, y cada versión queda fechada en el

historial del repositorio.

3.5.1 Definiciones operativas

Una métrica sin un evento de inicio y uno de fin medibles por una máquina no se puede reportar. La tabla 3.5 da los dos eventos de cada una. Todas las marcas salen de un mismo reloj: en simulación, el de la simulación; en los vehículos, el del computador que graba.

Tabla 3.5. Definición operativa de las cuatro métricas del anteproyecto.

Métrica	**Inicio**	**Fin**
Tiempo de respuesta	El coordinador acepta la solicitud	Primera de tres muestras seguidas de la odometría del vehículo asignado con velocidad de al menos 0,02 m/s
Tiempo de asignación	Primera publicación de la etapa recibida	Primera publicación del primer tramo, con el agente asignado
Tasa de éxito	Proporción de misiones exitosas sobre las corridas válidas	
Continuidad entre niveles	El primer vehículo alcanza su punto de transferencia	Primer movimiento del segundo vehículo

Una misión es exitosa si cumple tres condiciones. La pose final del vehículo, en su odometría, está a 0,25 m o menos del destino declarado en el catálogo; en los vehículos el umbral es de 0,5 m. La misión llegó a la etapa completada sin pasar por fallida. Y, si era entre niveles, hubo un relevo.

La regla que consideramos central en el protocolo es que el éxito se mide contra la odometría del vehículo, nunca contra el resultado que devuelve Nav2. Una navegación puede terminar en éxito y haber dejado al vehículo lejos del destino: en las primeras pruebas en simulación, una de tres metas devolvió éxito con el vehículo a 0,296 m. Aceptar ese resultado sería dejar que el sistema se evalúe a sí mismo.

El tiempo de asignación resultó menor que el paso de 100 ms del reloj de la simulación, así que se reporta como una cota por misión y se caracterizó aparte, en un banco de pruebas con el reloj del sistema. La continuidad se definió primero como una condición binaria, que resultó no poder fallar, y se reformuló como explica la sección 4.4.

3.5.2 Diseño de la campaña en simulación

La campaña en simulación tiene treinta misiones: quince con origen y destino en el mismo piso, que sirven de control, y quince entre pisos, que llevan un relevo y responden la pregunta de investigación. Con diez misiones corridas se vio que las misiones entre pisos navegaban siempre por el piso 2 y las de control no, de modo que no se habría podido separar el efecto del relevo del efecto del piso. El diseño se enmendó para equilibrar el piso dentro de cada grupo, en cuatro estratos de ocho, siete, ocho y siete misiones, y la enmienda quedó escrita en el protocolo.

Los pares de origen y destino se sortean con una semilla fija, publicada antes de correr, porque con pares elegidos a mano se corre el riesgo de probar justo los recorridos que funcionan. Una corrida fallida cuenta como fallo. Solo se descarta si la causa es ajena al sistema y está en una lista cerrada, escrita antes de la campaña, que admite la caída del simulador, un controlador del vehículo que no se activa, el factor de tiempo real por debajo de 0,99, un fallo de la máquina anfitriona o una cancelación por una causa externa anotada antes de conocer el resultado. Si los descartes superan el 20 % de las corridas, la campaña no es válida.

3.5.3 Instrumentación

Cada misión deja una grabación de datos de ROS (*rosbag*) con los tópicos del coordinador y de los dos vehículos. Un programa compone a partir de ella el registro de la misión, en un formato estructurado que se valida contra un esquema versionado. El registro contiene las marcas de cada etapa, la trayectoria de cada vehículo, la pose final, el error de llegada y su procedencia. Un segundo programa comprueba la integridad de los registros y calcula las cuatro métricas con sus intervalos de confianza. Se escribió antes de la campaña, y encontró dos defectos que después habrían sido irreparables: la continuidad no tenía campo en el esquema, y el intervalo que citaba el protocolo no se reproducía con el método declarado.

Con los vehículos, el grabador corre en el contenedor con Jazzy, y un programa adapta la grabación al formato que leen las herramientas de Humble. El registro usa el reloj del computador, la tolerancia de 0,5 m y el mapa de cada piso. La llegada la verifica el coordinador con la pose que publica el agente, 1,5 s después de que Nav2 la da por alcanzada. Nav2 la declara en cuanto el vehículo entra en su margen, y el vehículo sigue rodando. En las llegadas medidas en el piso 4, el vehículo estaba a entre 0,53 y 0,62 m de la meta en la última orden de avance, y un segundo después, ya quieto, a entre 0,09 y 0,53 m. La distancia de llegada se guarda con su procedencia, medida con flexómetro o tomada de la localización del vehículo, y el analizador advierte cuando no hay medida externa, como ocurrió en la campaña con los dos vehículos (sección 4.4.3).

3.5.4 Pruebas de verificación de la demostración física

La demostración sobre los vehículos se organizó en seis pruebas de verificación sucesivas, con criterios propuestos por los autores y decididos por el director. Cada prueba tiene un criterio escrito antes de correrla y lo que significa no superarla (tabla 3.6). Se fijaron tres fechas de revisión en octubre, y después de la última, el 16 de octubre, no se toman más datos.

Tabla 3.6. Pruebas de verificación de la demostración física y su estado al cerrar la toma de datos.

Prueba	**Criterio**	**Estado**
Actuación	El vehículo responde a las órdenes de dirección y tracción	Superada
Odometría	Error de 10 % o menos sobre un recorrido conocido de 5 m o más	Superada
Navegación de un vehículo	Un vehículo llega de un punto a otro con Nav2, con la llegada verificada	Superada
Dos vehículos en la misma red	Los dos vehículos y el coordinador coexisten sin conflicto de nombres	Superada
Misión con relevo	Una misión con relevo sobre los dos vehículos, con su registro	Superada
Campaña con los vehículos	Entre 5 y 10 repeticiones del protocolo con su registro	Superada

Para la campaña con los vehículos se sortearon, antes de correr y con una semilla fija, diez misiones entre los pisos 3 y 4: cinco de subida y cinco de bajada. Se corrieron en ese orden y sin reubicar los vehículos entre misiones, como ocurriría en la operación real. Sus resultados están en la sección 4.3.

4. RESULTADOS

Los resultados llegan de tres fuentes. La primera es la campaña en simulación, con treinta misiones sorteadas (sección 4.1). La segunda son las pruebas de verificación de la plataforma física, que miden la odometría y la navegación de un vehículo (sección 4.2). La tercera es el sistema completo sobre los dos vehículos, con una misión con relevo y una campaña de misiones sorteadas entre los pisos 3 y 4 (sección 4.3). Las dos campañas se reportan por separado. La de simulación produce la evidencia estadística, y la de los vehículos demuestra que el sistema opera sobre la plataforma real.

4.1 Evaluación en simulación

La campaña corrió treinta misiones sorteadas: quince de control, con origen y destino en el mismo piso, y quince entre pisos, con un relevo. El analizador la dictaminó válida, sin ningún descarte, frente a un máximo admitido del 20 %.

4.1.1 Tasa de éxito

Una misión es exitosa si la pose final queda a 0,25 m o menos del destino, medida contra la odometría y no contra el resultado de Nav2, y si llega a completarse sin pasar por fallo. La tabla 4.1 da la tasa de cada grupo con su intervalo de confianza de Wilson al 95 % [52].

Tabla 4.1. Tasa de éxito de la campaña en simulación, con intervalo de confianza de Wilson al 95 %.

Misiones	n	éxitos	tasa	intervalo al 95 %
Campaña completa	30	26	86,7 %	70,3 – 94,7 %
Mismo piso (control)	15	12	80,0 %	54,8 – 93,0 %
Entre pisos, con relevo	15	14	93,3 %	70,2 – 98,8 %

Lo que podemos afirmar es que la tasa de éxito supera el 70 %, no que sea del 86,7 %. Con treinta misiones el intervalo mide 24 puntos porcentuales de ancho, y ese límite es de diseño: solo se reduce con más muestra. El 86,7 % es el estimador puntual y se reporta siempre junto a su intervalo. Los intervalos de los dos grupos se solapan, así que la comparación entre ellos no sostiene ninguna conclusión, aunque el estimador puntual apunte en contra de la intuición: las misiones con relevo salieron mejor que las de control.

4.1.2 Causa común de los cuatro fallos

Las cuatro misiones fallidas quedaron a 0,295, 0,284, 0,311 y 0,347 m del destino, frente a una tolerancia de 0,25 m. Las cuatro se quedaron cortas: ninguna se pasó del destino ni terminó en otro sitio. Los cuatro fallos se deben al límite de precisión de la aproximación final, y ninguno a la coordinación. El error de llegada del conjunto tiene una mediana de 0,106 m y va de 0,019 a 0,347 m.

4.1.3 Tiempo de respuesta y tiempo de asignación

El tiempo de respuesta se reporta por escalones, y no con media y desviación, por una razón de instrumento (tabla 4.2).

Tabla 4.2. Tiempo de respuesta de las treinta misiones, en escalones del reloj de la simulación.

Tiempo de respuesta	0,1 s	0,2 s	0,3 s	0,4 s
Misiones	10	15	4	1

Los cuatro valores son múltiplos exactos de 100 ms, el paso con que avanza el reloj de la simulación, y la resolución del instrumento vale entre el 25 % y el 100 % de la magnitud medida. Una mediana de 0,2 s significa «uno o dos pasos del reloj», no «doscientos milisegundos», y dar media y desviación sugeriría una precisión que el reloj no entrega. Lo que sí se sostiene es la cota, que además es lo que le importa a un usuario: el robot arranca en menos de medio segundo.

El tiempo de asignación quedó por debajo de 100 ms en las treinta misiones, varios órdenes de magnitud por debajo de un paso del reloj. Por eso se caracterizó aparte, en un banco de pruebas con el reloj del sistema: mediana entre 154 y 175 µs y máximo de 306 µs. La figura 4.1 muestra una misión de la campaña.

4.2 Evaluación con los vehículos

La demostración física se organizó en pruebas de verificación sucesivas (sección 3.5.4). Esta sección reporta las dos que producen cifras: la de odometría y la de navegación de un vehículo.

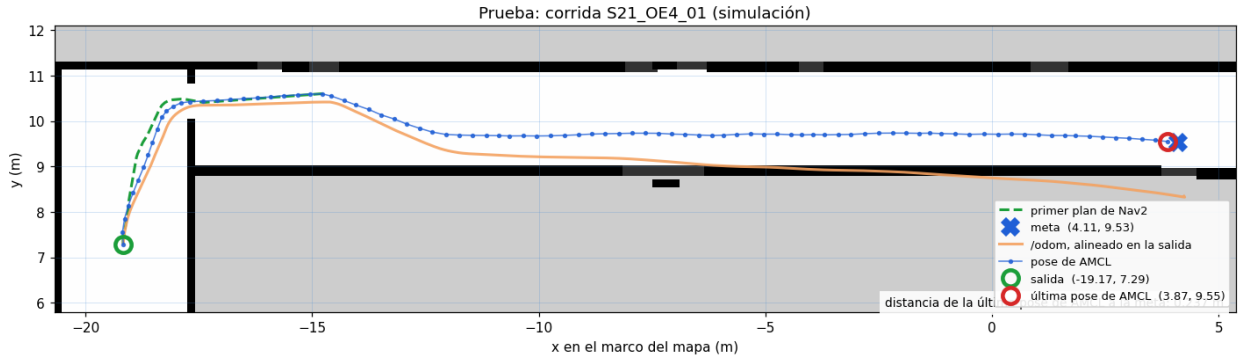


Figura 4.1. Una misión de la campaña en simulación. Se superponen el primer plan que calcula el navegador, la pose estimada a lo largo del recorrido y la odometría cruda. La separación creciente entre las dos últimas es la deriva que la localización corrige contra el mapa.

4.2.1 La odometría del vehículo

La odometría del vehículo sale de comparar barridos sucesivos del láser, y durante meses no produjo una medida utilizable. La tabla 4.3 reúne lo que se midió en cada entorno, como fracción del avance real medido con flexómetro.

Tabla 4.3. Fracción del avance real que registra la odometría, por entorno. El criterio de la prueba es un error de 10 % o menos.

Entorno	Recorrido	Registrado	Criterio
Pasillo simulado, 46,9 m	largo	37,9 %	no cumple
Pasillos de los pisos 1 y 2	largo	5,7 % y 1,3 %	no cumple
Hall del piso 2	6 m	73 %	no cumple
Cuarto cerrado, desplazado a mano	3 m	96,3 – 101,9 %	cumple
Pasillo del piso 4	6,68 m	95,4 %	cumple
Pasillo del piso 4	14,57 m	103,3 %	cumple
Pasillo del piso 4, con la unidad inercial	8,08 m	101,9 %	cumple
Pasillo del piso 4, con la unidad inercial	2,82 m	104,9 %	cumple

El estimador es el mismo en todas las filas; lo que cambió fue el entorno. En un pasillo recto y uniforme la información de avance no está en la señal del sensor, y ningún ajuste la recupera (sección 2.3.4). En un pasillo con hornacinas, casilleros y quiebres, con ocho cambios de perfil en 25 m, la misma odometría queda dentro del ± 5 % sin cambiar un parámetro. Con las dos mediciones del pasillo del piso 4, de 6,68 y 14,57 m, el director dio por superada la prueba de odometría, de la que dependía todo lo demás sobre los vehículos. Las dos últimas filas ya incluyen la unidad inercial (sección 3.3.6); el segundo de esos tramos mide 2,82 m, por debajo de los 5 m que pide el criterio, y se incluye como dato de la misma cadena.

4.2.2 La navegación y la precisión de llegada

La prueba de navegación de un vehículo pide un recorrido de punto a punto con Nav2 y la llegada verificada con flexómetro, a 0,5 m o menos del destino. Es la tolerancia que fijó el director para los vehículos reales. En la primera sesión en el piso 4 las dos llegadas medidas quedaron a 0,57 y 0,62 m, cortas, porque Nav2 daba entonces la meta por alcanzada a 1,0 m. Con la unidad inercial integrada (sección 3.3.6) ese margen bajó a 0,5 m. En la sesión siguiente, uno de los vehículos recorrió en el piso 4 tres tramos encadenados, sin que nadie lo tocara entre uno y otro, y después regresó a la salida (tabla 4.4 y figura 4.2).

Tabla 4.4. Cadena de tres tramos en el piso 4 y regreso a la salida: llegada de cada tramo.

Tramo	Destino	Tiempo	Recuperaciones	A la meta	Medida
1	Salón 403	13,5 s	1	0,26 m	estimada
2	Salón 402	27,8 s	1	0,45 m	flexómetro
3	Salón 401	5,2 s	0	0,06 m	flexómetro
Regreso	Salida	62,8 s	6	0,34 m	flexómetro

Las llegadas medidas quedaron dentro de la tolerancia de 0,5 m, y el director dio por superada la prueba. La llegada al Salón 403 no se midió porque en ese momento no había flexómetro, y se estimó con el error de los otros dos tramos. El regreso a la salida incluyó una media vuelta en el tramo ancho frente a los salones, que Nav2 resolvió con seis recuperaciones.

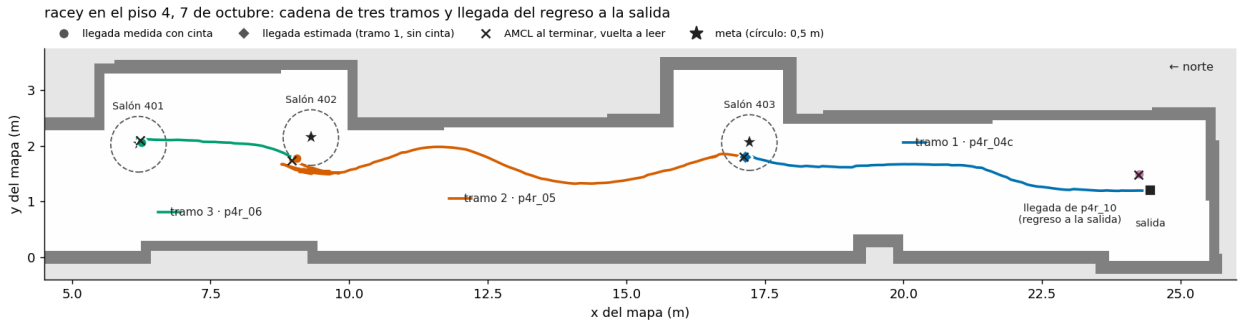


Figura 4.2. Cadena de tres tramos en el piso 4 y regreso a la salida: recorrido según la localización, llegadas medidas con flexómetro, la llegada estimada al Salón 403 y la pose de la localización al terminar cada tramo.

Al día siguiente, el otro vehículo recorrió el piso 3 en cinco tramos encadenados, con dos medias vueltas entre ellos, y los cinco terminaron con éxito y sin recuperaciones. Esas llegadas no se midieron con flexómetro, y al forzar a la localización a recalcular su pose al final de cada tramo, esta se desplazó entre 0,61 y 0,86 m. La pose que publica la localización al detenerse el vehículo va

atrasada, y por eso la llegada que vale es la medida con flexómetro. Lo que condiciona la precisión de llegada sigue siendo la banda muerta de tracción (sección 3.3.5): el vehículo se aproxima a la meta a 0,40 m/s y no tiene régimen de aproximación fina.

4.3 Evaluación del sistema completo con los dos vehículos

El sistema completo funcionó sobre los dos vehículos. La misión con relevo del Salón 302 al Salón 402 se completó, y de las diez misiones sorteadas para la campaña, las nueve que llegaron a correr se completaron, todas con relevo. La décima quedó interrumpida porque el computador portátil se apagó por falta de batería.

4.3.1 Condiciones de las pruebas

Las pruebas se hicieron en los pisos 3 y 4, con un vehículo en cada piso. El coordinador y el grabador corrieron en el portátil, y cada misión se pidió desde la página web en un teléfono conectado a la red de los vehículos. Los autores hicieron de usuario: pidieron cada misión, acompañaron a los vehículos y confirmaron el cambio de piso en la página. Los vehículos no se reubicaron entre misiones, y cuando el destino quedaba detrás del vehículo, el coordinador pidió la media vuelta en varios tiempos (sección 3.3.8). La llegada se verificó con la localización de cada vehículo y no con flexómetro, y la sección 4.4.3 explica cómo se lee eso.

4.3.2 La misión con relevo

La primera prueba del sistema completo fue una misión del Salón 302, en el piso 3, al Salón 402, en el piso 4. Se completó en 132,5 s, con un relevo y con la llegada a 0,444 m del destino. Entre la llegada del primer vehículo a la escalera y el arranque del segundo pasaron 61,1 s. Fue el segundo intento: en el primero, el coordinador rechazó la llegada del vehículo del piso 3 al Salón 302, adonde iba a recoger al usuario, porque su localización lo situaba a 0,59 m del punto.

Esa sesión sirvió de ensayo del montaje, y en ella los autores eligieron las misiones sin seguir el sorteo. De las doce misiones grabadas, siete se completaron y cinco terminaron fallidas: en cuatro, el coordinador rechazó una llegada con el vehículo a entre 0,51 y 0,78 m del punto, y en la otra Nav2 abortó la navegación. Estas misiones no forman parte de la campaña.

4.3.3 La campaña con las misiones sorteadas

Las diez misiones de la campaña se sortearon antes de correr, con una semilla fija, entre los puntos de interés de los dos pisos: cinco de subida y cinco de bajada (sección 3.5.4). Corrieron en el orden del sorteo, y la tabla 4.5 da el resultado de las nueve que llegaron a correr.

Tabla 4.5. Campaña con los dos vehículos: las nueve misiones sorteadas que corrieron, todas completadas con un relevo. La llegada es la de la localización del vehículo, con una tolerancia de 0,5 m.

N.º	Origen	Destino	Tiempo total	Respuesta	Llegada
1	Ascensor, piso 3	Salón 403	52,3 s	0,51 s	0,435 m
2	Salón 302	Salón 401	90,1 s	0,27 s	0,202 m
3	Salón 401	Salón 302	87,6 s	3,90 s	0,143 m
4	Salón 403	Salón 303	78,9 s	1,25 s	0,420 m
5	Salón 402	Salón 301	105,4 s	0,66 s	0,285 m
6	Salón 301	Salón 401	89,8 s	2,97 s	0,226 m
7	Ascensor, piso 4	Salón 301	81,7 s	0,87 s	0,234 m
8	Salón 302	Salón 403	81,8 s	0,95 s	0,214 m
9	Salón 303	Salón 402	92,6 s	0,63 s	0,299 m
Mediana			87,6 s	0,87 s	0,234 m

Con nueve éxitos en nueve misiones, el intervalo de Wilson al 95 % va del 70,1 al 100 %. Lo que podemos afirmar es que la tasa de éxito sobre los vehículos supera el 70 %, la misma cota que sostiene la campaña en simulación. Expresado como en la sección 2.3.7, nueve repeticiones sin fallo demuestran una fiabilidad mínima de 0,81 con un 85 % de confianza.

El error de llegada tuvo una mediana de 0,234 m y fue de 0,143 a 0,435 m. El tiempo total, desde la solicitud hasta la llegada verificada, tuvo una mediana de 87,6 s y fue de 52,3 a 105,4 s. El tiempo de respuesta, desde que el coordinador acepta la solicitud hasta que el vehículo asignado empieza a moverse, tuvo una mediana de 0,87 s y quedó por debajo de 1,3 s en siete de las nueve misiones. Los dos valores más altos, 3,90 y 2,97 s, son de misiones en que el vehículo ya estaba en el punto de origen. El coordinador verificó esa llegada, que no exigía moverse, con la espera de 1,5 s, y el primer movimiento fue la media vuelta hacia la escalera. El tiempo de asignación fue de 0,3 ms en las nueve misiones, medido con el reloj del portátil e incluyendo el paso del mensaje hasta el grabador, del mismo orden que el del banco de pruebas (sección 4.1). En las nueve misiones los vehículos hicieron 24 medias vueltas, de entre 160 y 171°, en uno a cuatro tiempos cada una.

La figura 4.3 descompone el tiempo de cada misión en sus etapas. Ir a recoger al usuario tomó entre 2,2 y 30,6 s, según dónde había terminado la misión anterior. El guiado en el piso de origen tuvo una mediana de 24,2 s, y el guiado en el piso de destino, de 24,8 s.

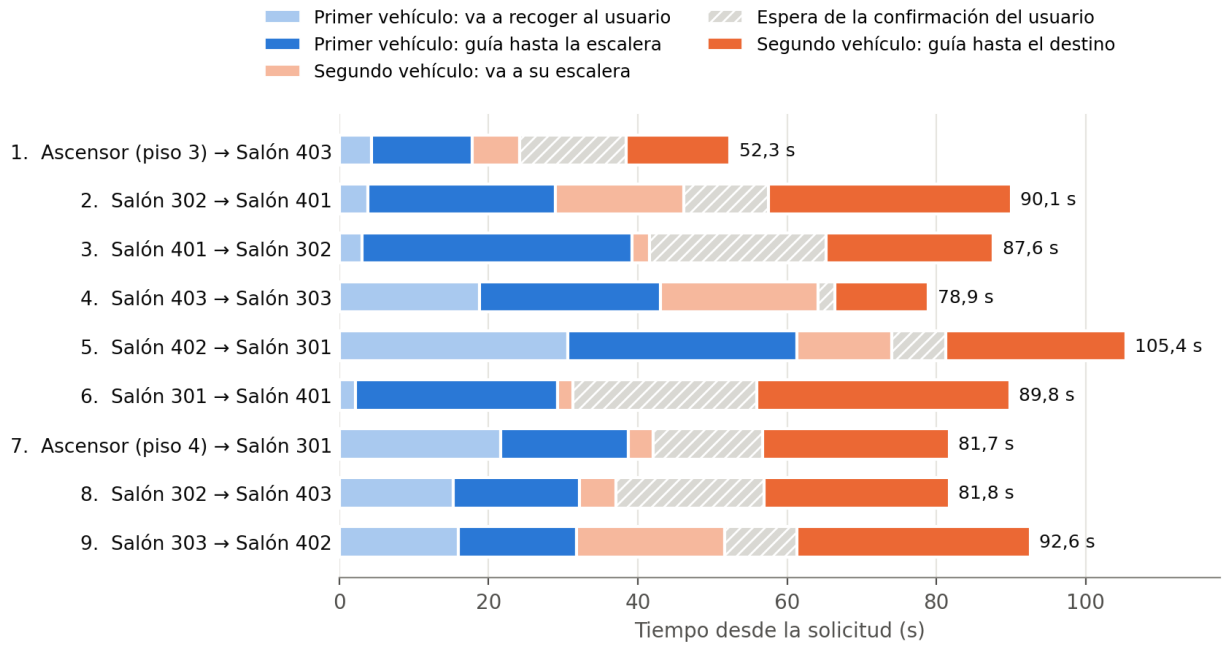


Figura 4.3. Duración de cada etapa en las nueve misiones de la campaña con los vehículos, según las marcas de etapa del coordinador. El tramo rayado es la espera de la confirmación de cambio de piso, contada desde que el segundo vehículo está en su escalera.

4.3.4 El relevo entre los dos vehículos

En las nueve misiones el relevo se produjo y el coordinador no pasó por el estado inactivo ni quedó sin agente asignado. Como en simulación, ese criterio no puede fallar por la forma en que está construido (sección 4.4). La medida que informa es el intervalo del relevo: el tiempo desde que el primer vehículo llega a su escalera hasta el primer movimiento del segundo.

Ese intervalo tuvo una mediana de 1,22 s, pero la cifra junta dos situaciones. En cinco misiones el segundo vehículo tuvo que desplazarse hasta su escalera, en cuatro desde el destino de la misión anterior y en una para corregir su posición unos centímetros, y arrancó en menos de 1,3 s. En las otras cuatro ya estaba en la escalera, su primer movimiento fue el del guiado final, después de la confirmación, y el intervalo, de 12,9 a 27,5 s, mide sobre todo cuánto tardó la persona en confirmar.

Para la persona, lo que cuenta es el tiempo entre la llegada a la escalera del piso de origen y el arranque del guiado en el piso de destino. En ese tiempo cambia de piso, el segundo vehículo llega a su escalera si no estaba allí, y la persona confirma en la página. Tuvo una mediana de 24,8 s y fue de 18,0 a 29,5 s. La confirmación, desde que el segundo vehículo estaba en su escalera, tomó entre 2,3 y 24,7 s, y el aviso de los 60 s no llegó a mostrarse en ninguna misión.

4.3.5 Criterio de conteo e incidencias

La campaña cuenta, por decisión de los autores, una corrida por misión: la que se completó. El protocolo, en cambio, cuenta como fallo toda corrida fallida, y por eso los intentos que no entran en la tasa se declaran aquí. Las misiones 1 y 2 se completaron en el segundo intento. Los dos primeros intentos fallaron en el tramo en que el vehículo del piso 3 iba a recoger al usuario. En el de la misión 1, el coordinador rechazó la llegada al ascensor, con el vehículo a 0,526 m del punto, y en el de la misión 2 no tenía la pose del vehículo para verificar la llegada.

Antes de esos intentos, la navegación de los dos vehículos dejó de responder. Ocho intentos de la misión 1 terminaron rechazados o abortados, con las transformadas de posición detenidas dentro de Nav2. Los dos vehículos llevaban horas encendidos, con cargas de 10 a 14 en procesadores de dos núcleos. Se reiniciaron las cadenas de navegación con la pose de salida, y después las nueve misiones corrieron sin tocar los vehículos. La causa de la detención no se estableció.

La misión 10 empezó y no terminó: el portátil se apagó por falta de batería unos 30 s después de pedirla, cuando el vehículo del piso 4 ya había llegado a su escalera. Unos minutos antes se había pedido por error el ascensor del piso 4 como destino, porque los ascensores de los dos pisos aparecían en la página con el mismo nombre. Ese intento no es una misión del sorteo y no se cuenta. Después de la campaña, la página muestra el piso junto a los nombres que se repiten (sección 3.4.5).

4.4 Limitaciones de la medición

Tres límites afectan a cómo deben leerse los resultados anteriores: dos de la campaña en simulación y uno de la campaña con los vehículos. Los declaramos aquí porque callarlos sería presentar como medida lo que no lo es.

4.4.1 Naturaleza del criterio de continuidad entre niveles

La continuidad del servicio es la variable de respuesta principal del cuarto objetivo, y su criterio binario se cumplió en las treinta misiones. Ese resultado, sin embargo, no es evidencia. El criterio exige que, dentro de la ventana de la misión, el coordinador nunca esté inactivo ni tenga el agente sin asignar. Los dos únicos puntos del código que producen esos valores ocurren antes de que la ventana se abra. No existe ejecución posible que lo incumpla, y un criterio que no puede dar resultado negativo no aporta evidencia, por muchas veces que se verifique [53].

No quisimos corregirlo inventando una ruta de fallo, porque eso habría sido instrumentar el experimento para que pareciera más informativo. La continuidad se declaró como un invariante estructural del diseño, y la afirmación se apoyó en una magnitud que sí varía: el intervalo del relevo, entre el final del tramo de un vehículo y el inicio del tramo del otro. En las quince misiones entre pisos tuvo una mediana de 0,1 s y un máximo de 0,2 s, es decir, uno o dos pasos del reloj de la simulación.

4.4.2 Espacio desconocido marcado como libre en el mapa

El mapa con que corrió la campaña en simulación declaraba un umbral que hacía que 10 908 celdas desconocidas, el 14,5 % del mapa, se cargaran como espacio libre. Con ello quedaban desactivadas en la práctica las dos salvaguardas que debían impedir planificar sobre lo no explorado. Esto no invalida el veredicto de la campaña: los cuatro fallos fueron de precisión de llegada, no de trazado, y el pasillo por el que se navegó está libre en el mundo real. Lo que no se sabe es si alguna trayectoria cruzó una de esas celdas, algo comprobable sobre los registros conservados. El defecto se corrigió después de la campaña.

4.4.3 La llegada en los vehículos, sin medida externa

En la campaña con los vehículos la llegada no tuvo una medida independiente del sistema. El protocolo la toma con flexómetro sobre marcas en el piso, y en esa sesión no se midió. El error que se reporta es el de la localización del propio vehículo contra el punto del catálogo, leído 1,5 s después de que Nav2 diera la meta por alcanzada, con el vehículo ya detenido. Es la medida con que el coordinador acepta la llegada, así que el sistema verificó su llegada con su propia estimación, que es lo que la regla principal del protocolo procura evitar (sección 3.5). Los registros lo declaran con la procedencia de la medida, y el analizador lo advierte al calcular las métricas.

La única comparación entre la localización y el flexómetro en los vehículos es la de la cadena de tres tramos en el piso 4 (sección 4.2.2). Leída de nuevo al terminar cada tramo, la localización quedó a 0,10 m o menos de la medida en el sentido del avance y a 5 cm o menos de lado. En el piso 3, en cambio, forzar a la localización a recalcular su pose al final de cada tramo la desplazó entre 0,61 y 0,86 m. Por eso leemos el resultado de la campaña así: cada vehículo, según su propia localización, llegó a menos de 0,5 m del destino. Que la distancia real cumpliera la misma tolerancia no está medido.

CONCLUSIONES

El primer objetivo específico, modelar la arquitectura funcional del sistema, se cumplió. El sistema tiene un contrato de interfaces, treinta y seis requisitos con su prueba y una capa de coordinación que ejecuta el relevo entre niveles. La decisión que más valoramos fue escribir el planificador como función pura. Permitió verificar las 930 combinaciones de origen y destino sin simulador y encontrar defectos que una corrida no habría delatado, como planes que mandaban al mismo robot dos veces a la misma pose. La asignación de tareas es óptima por la estructura del problema, con un solo agente admisible por nivel, y el aporte del trabajo está en coordinar dos agendas que dependen una de otra durante el relevo.

El segundo objetivo, la plataforma robótica de dos vehículos, se cumplió. Nav2 quedó configurado para cinemática tipo automóvil (sección 3.3.2). La odometría cumplió su criterio con un error entre $-4{,}6\,\%$ y $+3{,}3\,\%$, y una cadena de tres tramos sin intervención llegó a entre 0,06 y 0,45 m de sus metas (sección 4.2.2). Los dos vehículos operan juntos en la misma red, con un tiempo de ida y vuelta de 5,99 ms de mediana, y en la campaña hicieron 24 medias vueltas en pasillos de 2,5 a 3,2 m. Quedan dos límites en la plataforma. La banda muerta de tracción impide una aproximación fina y obliga a una tolerancia de llegada de 0,5 m en lugar de 0,25 m (sección 3.3.5). Y la estabilidad en sesiones largas no está resuelta: tras horas encendidos, la navegación de los dos vehículos dejó de responder por una causa que no se estableció.

El tercer objetivo, la interfaz de interacción con el usuario, se cumplió. Es una página web que funciona en el navegador de un teléfono, sin instalación y sin salida a internet, y con ella se pidieron y siguieron todas las misiones de la campaña con los vehículos. A nuestro juicio, las dos funciones que se añadieron al ver el sistema en uso son las que más lo acercan a una operación real. La confirmación de cambio de piso impide dar por completada una misión con el usuario en el nivel equivocado, y la cancelación devuelve el vehículo a su escalera en cualquier etapa. Las pruebas con los vehículos dejaron además tres ajustes de la página, ya corregidos (sección 3.4.5).

El cuarto objetivo, la evaluación experimental, se cumplió en simulación y con los vehículos. La campaña de treinta misiones en simulación fue válida y permite afirmar, con $95\,\%$ de confianza, que la tasa de éxito supera el $70\,\%$. Con los dos vehículos, las nueve misiones sorteadas que corrieron se completaron con relevo, lo que sostiene la misma cota (sección 4.3), con una mediana de 87,6 s de tiempo total y de 0,87 s de respuesta. Su llegada se verificó con la localización de cada vehículo y no con una medida externa (sección 4.4.3). La evaluación nos dejó también una conclusión de método:

costó más averiguar qué medía cada prueba que construir el sistema. El reloj de la simulación no tenía resolución para el tiempo de asignación, el criterio de continuidad no podía fallar, y el modelo del edificio, que se daba por correcto, daba un hall un 25 % más ancho que el real. Los tres casos se encontraron preguntando si el criterio podía fallar, y ninguno habría aparecido corriendo más experimentos.

Sobre el objetivo general, concluimos que el esquema de relevo es viable. Dos robots dedicados cada uno a un nivel pueden repartirse el guiado de una persona que cambia de piso, sin que ninguno atraviese la zona de transición y sin modificar el edificio. Quedó demostrado en simulación, con una campaña medida y válida, y sobre los dos vehículos reales, que ejecutaron el relevo entre los pisos 3 y 4 en todas las misiones de su campaña. Lo que el trabajo no mide es la experiencia de la persona guiada, que queda para un estudio con usuarios.

TRABAJOS FUTUROS

Las líneas que siguen salen de límites concretos que el trabajo encontró y de las suposiciones que el diseño hizo para mantenerse dentro del alcance.

Un régimen lento y una medida de avance propia. El vehículo no se mueve por debajo de 0,40 m/s (sección 3.3.5), y su posición sale de comparar barridos del láser, que falla por construcción en pasillos uniformes (sección 2.3.4). Calibrar el variador o cambiar la relación de transmisión le daría un régimen lento. Codificadores en las ruedas, u odometría visual con las cámaras que el vehículo ya lleva, le darían una medida de avance que no depende del sitio. Con esa medida se podría cerrar el lazo de velocidad cerca de la meta, y el sistema dejaría de necesitar un entorno con estructura para operar.

Campañas más grandes y con medida externa. Con treinta misiones en simulación el intervalo de confianza mide 24 puntos porcentuales, y con nueve en los vehículos la cota inferior es del 70 %. Ese ancho solo se reduce con más repeticiones, que permitirían además comparar las misiones de un mismo piso con las de dos pisos, una comparación que hoy no se sostiene porque sus intervalos se solapan. La campaña física debería repetirse con la llegada medida con flexómetro o con un sistema de seguimiento externo (sección 4.4.3).

Estabilidad de la navegación en sesiones largas. Tras horas encendidos, la navegación de los dos vehículos dejó de responder y hubo que reiniciarla, por una causa que no se estableció (sección 4.3). Antes de dejar el sistema en servicio hace falta reproducir la falla en una sesión larga controlada. Con ella se puede decidir si basta con supervisar la cadena de navegación y reiniciarla sola, o si hay que bajar la carga del procesador.

El ascensor como segunda vía de transición. Hoy el cambio de piso se hace siempre por la escalera. Admitir el ascensor exige que el planificador elija entre dos puntos de transferencia por nivel, por cercanía o por preferencia del usuario. El planificador ya detecta ese caso y falla con un mensaje explícito, de modo que el punto de extensión está identificado.

Esquinas en pasillos estrechos. Todos los recorridos medidos fueron sobre tramos rectos. Falta probar que el vehículo resuelva un giro de 90° en una esquina estrecha, que es lo que exige el guiado en edificios con más de un pasillo por piso.

Más de un vehículo por nivel y misiones simultáneas. El sistema atiende una misión a la vez, con un solo vehículo por nivel, y con esas condiciones la asignación es trivial (sección 2.3.1). Con dos o más vehículos elegibles por tarea, o varias solicitudes a la vez, los métodos de subasta como MURDOCH [34] o CBBA [35] empiezan a decidir algo. Es la extensión natural del trabajo para edificios con más afluencia.

Evaluación con usuarios. Las cuatro métricas del proyecto miden el sistema, y la experiencia de quien lo usa queda sin medir. Un estudio con personas, como el que hicieron Guerreiro et al. [11] con su robot guía, permitiría medirla. Diría si la interfaz se entiende sin explicación, si la velocidad del vehículo es cómoda para caminar a su lado y cuánto tolera el usuario la espera en la escalera. Es también el paso previo a ofrecer el sistema a personas con discapacidad visual, que es la aplicación de más impacto que sugiere la literatura revisada.

REFERENCIAS BIBLIOGRÁFICAS

[1] M. Jiménez Muñoz, «Arquitectura cognitiva para integración de usuarios humanos dentro de un sistema de mapas multinivel en una plataforma robótica asistencial,» Tesis de mtría., Universidad Politécnica de Madrid, Dept. Ing. Industrial, Madrid, España, 2024.

[2] B. P. Gerkey y M. J. Matarić, «A Formal Analysis and Taxonomy of Task Allocation in Multi-Robot Systems,» *The International Journal of Robotics Research*, vol. 23, n.º 9, págs. 939-954, 2004. DOI: 10.1177/0278364904045564.

[3] G. T. Varghese et al., «Multi-Robot System for Mapping and Localization,» en *8th International Conference on Robotics and Automation Engineering (ICRAE)*, 2023, págs. 1-6. DOI: 10.1109/ICRAE59816.2023.10458504.

[4] T. Kim, G. Kang, D. Lee y D. H. Shim, «Development of an Indoor Delivery Mobile Robot for a Multi-Floor Environment,» *IEEE Access*, vol. 12, págs. 1-12, 2024. DOI: 10.1109/ACCESS.2024.3381489.

[5] Z. Song, S. Lou, R. Tan y C. Lv, «Robot-Elevator Interaction through IoT Devices for Autonomous Robot Multi-Floor Transition,» en *10th International Conference on Electrical Engineering, Control and Robotics (EECR)*, 2024, págs. 1-6. DOI: 10.1109/EECR60807.2024.10607245.

[6] S. Macenski, T. Foote, B. Gerkey, C. Lalancette y W. Woodall, «Robot Operating System 2: Design, Architecture, and Uses in the Wild,» *Science Robotics*, vol. 7, n.º 66, eabm6074, 2022. DOI: 10.1126/scirobotics.abm6074.

[7] K. Omer y A. Monteriu, «Multi-Layer Robotic Controller for Enhancing the Safety of Mobile Robot Navigation in Human-Centered Indoor Environments,» *Frontiers in Robotics and AI*, vol. 12, págs. 1-15, 2025. DOI: 10.3389/frobt.2025.00001.

[8] B. R. Nair, «Collaborative Perception in Multi-Robot Systems: Case Studies in Household Cleaning and Warehouse Operations,» en *6th International Conference on Robotics and Computer Vision (ICRCV)*, 2024, págs. 1-6. DOI: 10.1109/ICRCV60186.2024.00012.

[9] W. Burgard, A. B. Cremers, D. Fox, D. Hähnel, G. Lakemeyer, D. Schulz, W. Steiner y S. Thrun, «Experiences with an Interactive Museum Tour-Guide Robot,» *Artificial Intelligence*, vol. 114, n.º 1–2, págs. 3-55, 1999.

[10] D. Fox, W. Burgard, F. Dellaert y S. Thrun, «Monte Carlo Localization: Efficient Position Estimation for Mobile Robots,» en *Proceedings of the Sixteenth National Conference on Artificial Intelligence (AAAI)*, Orlando, FL, USA, 1999, págs. 343-349.

[11] J. Guerreiro, D. Sato, S. Asakawa, H. Dong, K. M. Kitani y C. Asakawa, «CaBot: Designing and Evaluating an Autonomous Navigation Robot for Blind People,» en *21st International ACM SIGACCESS Conference on Computers and Accessibility (ASSETS)*, Pittsburgh, PA, USA, 2019, págs. 68-82. DOI: 10.1145/3308561.3353771.

[12] A. Li y H. Gurocak, «Modified Bug Algorithm with Proximity Sensors to Reduce Human-Cobot Collisions,» en *9th International Conference on Automation, Robotics and Applications (ICARA)*, 2023, págs. 1-6. DOI: 10.1109/ICARA56516.2023.10126056.

[13] A. Sojan, C. Thomas, B. Raj, S. A. Simon, S. K. John y A. A. Jose, «Autonomous Indoor Navigator,» en *11th International Conference on Smart Computing and Communication (ICSCC)*, 2025, págs. 1-6. DOI: 10.1109/ICSCC66177.2025.11233830.

[14] S. Macenski, F. Martín, R. White y J. Ginés Clavero, «The Marathon 2: A Navigation System,» en *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, Las Vegas, NV, USA, 2020. arXiv: 2003.00368.

[15] S. Macenski, T. Moore, D. V. Lu, A. Merzlyakov y M. Ferguson, «From the Desks of ROS Maintainers: A Survey of Modern & Capable Mobile Robotics Algorithms in the Robot Operating System 2,» *Robotics and Autonomous Systems*, vol. 168, pág. 104 493, 2023. DOI: 10.1016/j.robot.2023.104493.

[16] M. Bojarski et al., «End to End Learning for Self-Driving Cars,» en *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, Preprint, arXiv:1604.07316, Daejeon, South Korea, 2016.

[17] L. H. Zain, «Real-Time Obstacle Avoidance for a Mobile Robot Using CNN-Based Sensor Fusion,» en *International Conference on Robotics and Automation (ICRA)*, Preprint, arXiv:2509.08095, 2025.

[18] J. López Velásquez, G. A. Acosta Amaya y J. A. Jiménez Builes, «Enhancing Mobile Robot Navigation: Integrating Reactive Autonomy through Deep Learning and Fuzzy Behavior,» *Journal of Intelligent & Robotic Systems*, vol. 110, n.º 2, págs. 45-58, 2024. DOI: 10.1007/s10846-024-02045-2.

[19] E. Roszkowska, P. Makowski-Czerski y L. Janiec, «Multi-Level Control for Multiple Mobile Robot Systems,» en *IEEE International Conference on Robotics and Automation (ICRA)*, 2023, págs. 1-7. DOI: 10.1109/ICRA48891.2023.10160734.

[20] B. Rudrusamy et al., «Multi-Robot Systems Using Broadcast Control Framework with Visual-Based Tracking,» en *7th International Conference on Mechanical Engineering and Robotics Research (ICMERR)*, 2022, págs. 1-6. DOI: 10.1109/ICMERR56497.2022.10097802.

[21] X. Xu, S. A. Prieto y B. García de Soto, «A Modular ROS Framework for Cooperative Multi-Robot Task Allocation in Digital Environments,» *MDPI Sensors*, vol. 25, n.º 4, págs. 1-20, 2025. DOI: 10.3390/s25040001.

[22] J. Zhan, X. Yu y T. Westerlund, «Mejorando la resiliencia de los sistemas multirrobóticos basados en ROS 2 con Kubernetes: un estudio de caso sobre el posicionamiento,» *MDPI Sensors*, vol. 25, n.º 16, pág. 5067, 2025. DOI: 10.3390/s25165067.

[23] G. A. Korsah, A. Stentz y M. B. Dias, «A Comprehensive Taxonomy for Multi-Robot Task Allocation,» *The International Journal of Robotics Research*, vol. 32, n.º 12, págs. 1495-1512, 2013. DOI: 10.1177/0278364913496484.

[24] A. K. Srivastava, J. M. Levin y P. Dames, «DELIVER: A System for Coordinated Multi-Robot Pickup and Delivery Using Relay Planning,» en *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2025, págs. 1-8.

[25] A. E. Piguave Toala y D. S. Ronquillo Manosalvas, *Diseño e implementación de un sistema multi-robot de código abierto para ambientes colaborativos en ROS2*, Proyecto integrador, Fac. Ing. Electrónica, Escuela Superior Politécnica del Litoral, Guayaquil, Ecuador, 2023.

[26] S. Luo, Q. Yu y C. Zhu, «"Elevator or Stairs": A Layered Reinforcement Learning Approach for Robot Multi-Floor Navigation in High-Traffic Scenarios,» en *International Conference on Advanced Computing and Intelligent Robotics Applications (ACIRA)*, 2025, págs. 1-8. DOI: 10.1109/ACIRA67680.2025.11334740.

[27] J. Chen et al., «LITE: A Learning-Integrated Topological Explorer for Multi-Floor Indoor Environments,» en *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2025, págs. 1-8. DOI: 10.1109/IROS60139.2025.11246317.

[28] Z. Gong et al., «Stairway to Success: An Online Floor-Aware Zero-Shot Object-Goal Navigation Framework via LLM-Driven Coarse-to-Fine Exploration,» *IEEE Robotics and Automation Letters*, vol. 11, n.º 3, págs. 1-8, 2026. DOI: 10.1109/LRA.2026.3655265.

[29] B. Kehoe, S. Patil, P. Abbeel y K. Goldberg, «A Survey of Research on Cloud Robotics and Automation,» *IEEE Transactions on Automation Science and Engineering*, vol. 12, n.º 2, págs. 398-409, 2015. DOI: 10.1109/TASE.2014.2376492.

[30] N. Tahir y R. Parasuraman, «Edge Computing and Its Application in Robotics: A Survey,» *MDPI Sensors*, vol. 25, n.º 2, págs. 1-25, 2025. DOI: 10.3390/s25020001.

[31] I. Malavolta et al., «Computation Offloading for Ground Robotic Systems Communicating over WiFi,» *Empirical Software Engineering*, vol. 28, n.º 3, págs. 1-35, 2023. DOI: 10.1007/s10664-022-10268-0.

[32] H. W. Kuhn, «The Hungarian Method for the Assignment Problem,» *Naval Research Logistics Quarterly*, vol. 2, n.º 1–2, págs. 83-97, 1955. DOI: 10.1002/nav.3800020109.

[33] R. G. Smith, «The Contract Net Protocol: High-Level Communication and Control in a Distributed Problem Solver,» *IEEE Transactions on Computers*, vol. C-29, n.º 12, págs. 1104-1113, 1980.

[34] B. P. Gerkey y M. J. Matarić, «Sold!: Auction Methods for Multirobot Coordination,» *IEEE Transactions on Robotics and Automation*, vol. 18, n.º 5, págs. 758-768, 2002.

[35] H.-L. Choi, L. Brunet y J. P. How, «Consensus-Based Decentralized Auctions for Robust Task Allocation,» *IEEE Transactions on Robotics*, vol. 25, n.º 4, págs. 912-926, 2009. DOI: 10.1109/TRO.2009.2022423.

[36] Object Management Group, «Data Distribution Service (DDS), Version 1.4,» Object Management Group, inf. téc. formal/15-04-10, 2015.

[37] R. Siegwart, I. R. Nourbakhsh y D. Scaramuzza, *Introduction to Autonomous Mobile Robots*, 2.ª ed. Cambridge, MA, USA: MIT Press, 2011.

[38] J. A. Reeds y L. A. Shepp, «Optimal Paths for a Car That Goes Both Forwards and Backwards,» *Pacific Journal of Mathematics*, vol. 145, n.º 2, págs. 367-393, 1990. DOI: 10.2140/pjm.1990.145.367.

[39] D. Dolgov, S. Thrun, M. Montemerlo y J. Diebel, «Path Planning for Autonomous Vehicles in Unknown Semi-structured Environments,» *The International Journal of Robotics Research*, vol. 29, n.º 5, págs. 485-501, 2010. DOI: 10.1177/0278364909359210.

[40] S. Macenski, M. Booker y J. Wallace, *Open-Source, Cost-Aware Kinematically Feasible Planning for Mobile and Surface Robotics*, 2024. arXiv: 2401.13078.

[41] D. V. Lu, D. Hershberger y W. D. Smart, «Layered Costmaps for Context-Sensitive Navigation,» en *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, Chicago, IL, USA, 2014, págs. 709-715.

[42] R. C. Coulter, «Implementation of the Pure Pursuit Path Tracking Algorithm,» Robotics Institute, Carnegie Mellon University, Pittsburgh, PA, USA, inf. téc. CMU-RI-TR-92-01, 1992.

[43] S. Macenski, S. Singh, F. Martín y J. Ginés, «Regulated Pure Pursuit for Robot Path Tracking,» *Autonomous Robots*, vol. 47, págs. 685-694, 2023. arXiv: 2305.20026.

[44] S. Thrun, W. Burgard y D. Fox, *Probabilistic Robotics*. Cambridge, MA, USA: MIT Press, 2005.

[45] M. Jaimez, J. G. Monroy y J. González-Jiménez, «Planar Odometry from a Radial Laser Scanner. A Range Flow-Based Approach,» en *IEEE International Conference on Robotics and Automation (ICRA)*, Stockholm, Sweden, 2016, págs. 4479-4485. DOI: 10.1109/ICRA.2016.7487647.

[46] J. Zhang, M. Kaess y S. Singh, «On Degeneracy of Optimization-Based State Estimation Problems,» en *IEEE International Conference on Robotics and Automation (ICRA)*, Stockholm, Sweden, 2016, págs. 809-816.

[47] S. Macenski e I. Jambrecic, «SLAM Toolbox: SLAM for the Dynamic World,» *Journal of Open Source Software*, vol. 6, n.º 61, pág. 2783, 2021. DOI: 10.21105/joss.02783.

[48] T. Moore y D. Stouch, «A Generalized Extended Kalman Filter Implementation for the Robot Operating System,» en *13th International Conference on Intelligent Autonomous Systems (IAS-13)*, Padova, Italy, 2014.

[49] N. Koenig y A. Howard, «Design and Use Paradigms for Gazebo, an Open-Source Multi-Robot Simulator,» en *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, Sendai, Japan, 2004, págs. 2149-2154.

[50] C. Crick, G. Jay, S. Osentoski, B. Pitzer y O. C. Jenkins, «Rosbridge: ROS for Non-ROS Users,» en *Robotics Research*, ép. Springer Tracts in Advanced Robotics, H. I. Christensen y O. Khatib, eds., vol. 100, Cham, Switzerland: Springer, 2017. DOI: 10.1007/978-3-319-29363-9_28.

[51] L. D. Brown, T. T. Cai y A. DasGupta, «Interval Estimation for a Binomial Proportion,» *Statistical Science*, vol. 16, n.º 2, págs. 101-133, 2001. DOI: 10.1214/ss/1009213286.

[52] E. B. Wilson, «Probable Inference, the Law of Succession, and Statistical Inference,» *Journal of the American Statistical Association*, vol. 22, n.º 158, págs. 209-212, 1927. DOI: 10.1080/01621459.1927.10502953.

[53] K. R. Popper, *The Logic of Scientific Discovery*. London, UK: Hutchinson, 1959.

[54] ASTM International, «Standard Test Method for Navigation: Defined Area,» ASTM International, West Conshohocken, PA, USA, inf. téc. ASTM F3244-21, 2021.

[55] Departamento Nacional de Planeación, «Documento CONPES 3975: Política Nacional para la Transformación Digital e Inteligencia Artificial,» Consejo Nacional de Política Económica y Social, Bogotá D.C., Colombia, inf. téc., 2019.

[56] Departamento Nacional de Planeación, «Documento CONPES 4069: Política Nacional de Ciencia, Tecnología e Innovación 2022–2031,» Consejo Nacional de Política Económica y Social, Bogotá D.C., Colombia, inf. téc., 2021.

[57] ICONTEC, «NTC 1486: Documentación. Presentación de tesis, trabajos de grado y otros trabajos de investigación,» Instituto Colombiano de Normas Técnicas y Certificación, Bogotá D.C., Colombia, inf. téc., 2022.

[58] Donkey Car Project, *Donkey Car: an open source DIY self driving platform for small scale cars*, `https://www.donkeycar.com`, Consultado en abril de 2026.

[59] M. O'Kelly, V. Sukhil, H. Abbas et al., *F1/10: An Open-Source Autonomous Cyber-Physical Platform*, arXiv:1901.08567, 2019.

[60] AWS DeepRacer, *aws-deepracer: simulation artifacts, core components and configuration files to integrate the AWS DeepRacer device with the ROS Navigation stack*, Repositorio de GitHub, `https://github.com/aws-deepracer/aws-deepracer`, 2021.

[61] AWS DeepRacer Community, *deepracer-custom-car: custom software stack for the AWS DeepRacer*, Repositorio de GitHub, `https://github.com/aws-deepracer-community/deepracer-custom-car`, Consultado en mayo y agosto de 2026.

[62] Nav2 Project, *First-Time Robot Setup Guide*, Documentación de Nav2, `https://docs.nav2.org`, Consultado en septiembre de 2026.